

Project-3

project-3 (Web & Network Security VAPT Audit)

By- Dhruv Sharma

Email- Dhruv15saraswat@gmail.com

Introduction

Purpose of the Project

The primary purpose of this project is to establish a controlled, simulated environment for practical training and hands-on experience in cybersecurity. This lab replicates real-world scenarios in which a security professional identifies, analyzes, and demonstrates the exploitation of common vulnerabilities. Through this process, the project aims to build proficiency in both Network VAPT and Web Application VAPT, while fostering a deeper understanding of offensive security techniques and the countermeasures used to defend against them.

Network & Web Application Security — An Overview

Network Security refers to the practice of protecting a computer network from unauthorized access, misuse, modification, or disruption of its resources. Within the scope of this VAPT project, this involves enumerating hosts and open ports across a network, identifying the services running on those ports, and systematically testing them for weaknesses. Key areas of focus include SSL/TLS misconfigurations, default or easily brute-forced credentials, exploitation of vulnerable network services (such as SMB, FTP, and SSH), and the use of automated vulnerability scanning tools like OpenVAS and Nikto. The overarching objective is to evaluate the overall security posture of the network infrastructure.

Web Application Security focuses on protecting applications accessible over the Internet or an intranet — an increasingly critical discipline given that web applications represent one of the most frequently targeted attack surfaces. This project adopts the **OWASP Top 10** as its guiding framework, which catalogues the ten most critical security risks facing web applications today. Testing covers vulnerabilities that could allow attackers to steal sensitive data, hijack user accounts, or gain unauthorized administrative access to backend systems. Key vulnerability classes assessed include:

- SQL Injection
 - Cross-Site Scripting (XSS)
 - Broken Authentication
 - Insecure Direct Object References (IDOR)
 - Security Misconfigurations
 - Directory Traversal
 - Command Injection
 - File Upload vulnerabilities
-

Target Application and Lab Environment

Lab Environment

To ensure safe and isolated testing without impacting live systems, a dedicated VAPT lab environment has been constructed using virtualization technology:

Component	Details
Hypervisor	Vmware
Attacker Machine	Kali Linux (pre-configured with security tooling)
Vulnerable Server	Ubuntu Server With Laravel Vul App

Key tools deployed on the attacker machine:

Tool	Purpose
Nmap	Network discovery and security auditing
Burp Suite & OWASP ZAP	Intercepting and analyzing web application traffic
Metasploit	Developing and executing exploit code against targets
SQLMap	Automated detection and exploitation of SQL injection flaws
Nikto	Web server vulnerability scanning
Hydra	Parallelized brute-force login attacks

Target Applications

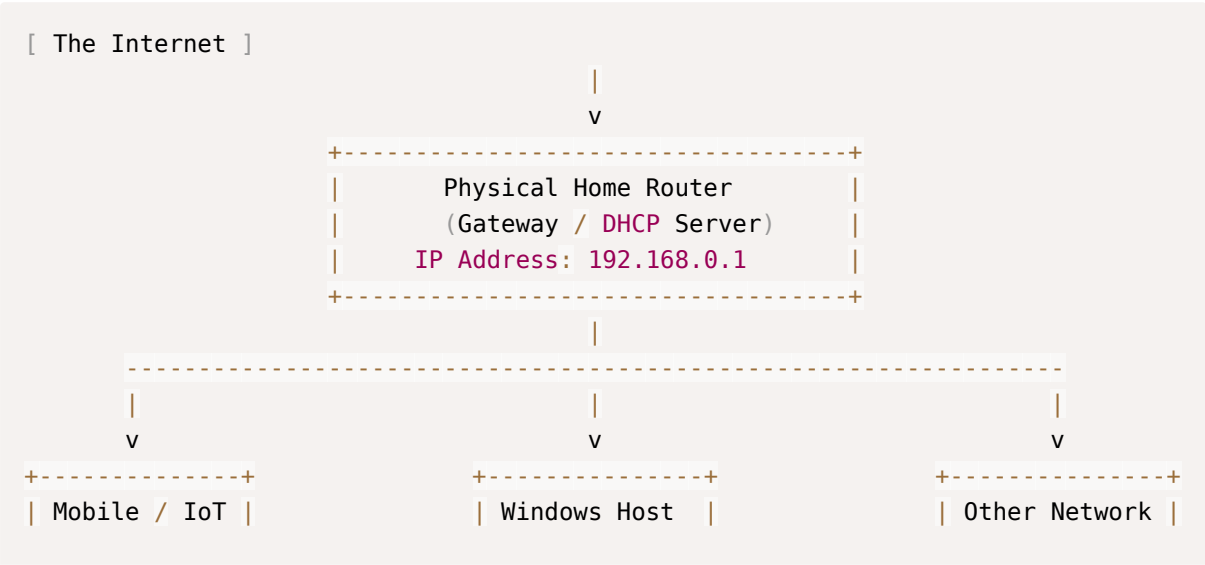
Testing is conducted against deliberately vulnerable applications, each designed to expose a broad spectrum of network and web application security weaknesses:

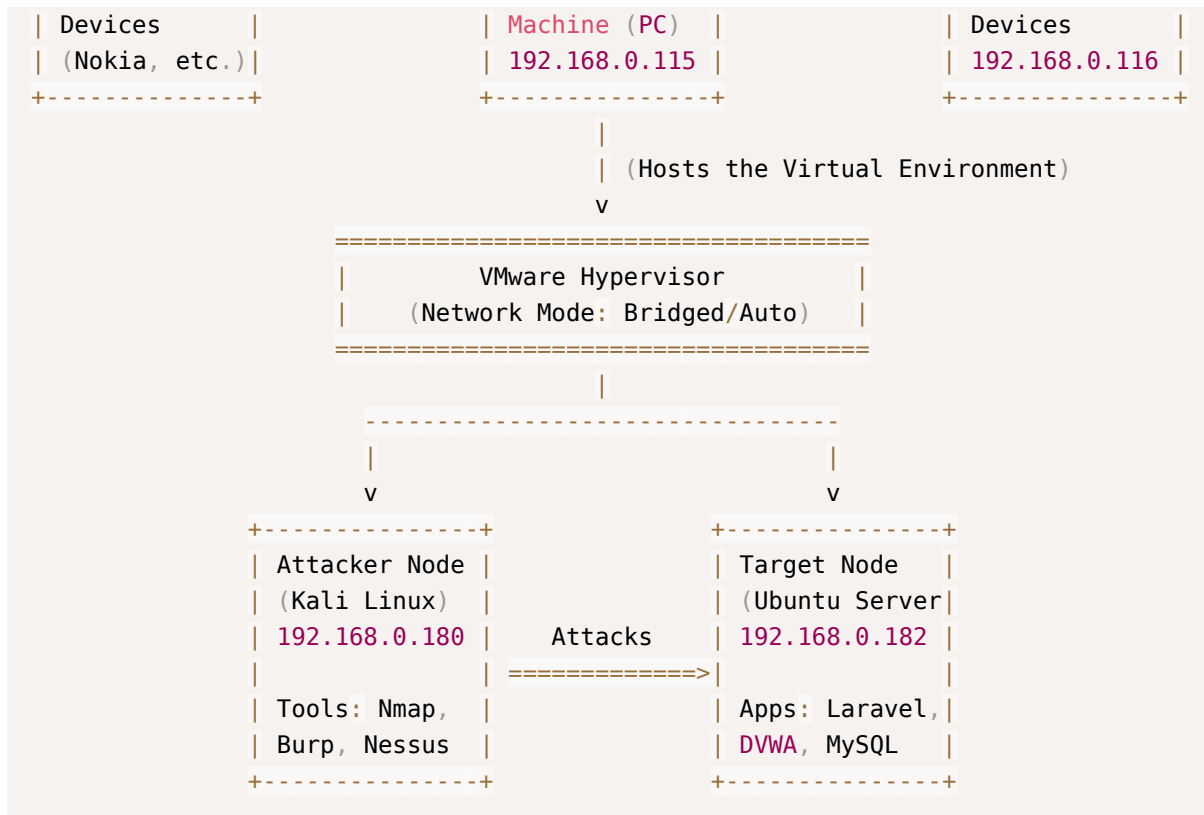
Target	Type	Description
Metasploitable2	Network	A Linux VM intentionally configured with network-level vulnerabilities
DVWA	Web Application	A PHP/MySQL application built to be deliberately insecure
OWASP Juice Shop	Web Application	A modern vulnerable app built on Node.js, Express, and Angular
WebGoat	Web Application	An insecure J2EE application developed and maintained by OWASP

The tables and structured layout should make this significantly cleaner for a formal submission. Let me know if you'd like to adjust the tone, expand any section, or reformat it for a Word document.

Lab Setup:

Network Topology





Ubuntu Server

step 1

install ubuntu 18.004.6

https://releases.ubuntu.com/?_ga=2.149898549.2084151835.1707729318-1126754318.1683186906&_gl=1*1fjf5wt*_gcl_au*MjAyMTAyNjk3Ny4xNzc0OTYwMDYw

step 2

setup vsftpd

```

machineone@vulnmachine:/var/www/html/web-app/public$ sudo apt install vsftpd
Reading package lists... Done
Building dependency tree
Reading state information... Done
vsftpd is already the newest version (3.0.3-9build1).
0 upgraded, 0 newly installed, 0 to remove and 45 not upgraded.
machineone@vulnmachine:/var/www/html/web-app/public$ sudo systemctl start vsftpd
machineone@vulnmachine:/var/www/html/web-app/public$ sudo systemctl enable vsftpd
Synchronizing state of vsftpd.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable vsftpd
machineone@vulnmachine:/var/www/html/web-app/public$ sudo apt install telnetd -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libfile-copy-recursive-perl openbsd-inetd tcpd update-inetd
The following NEW packages will be installed:
  libfile-copy-recursive-perl openbsd-inetd tcpd telnetd update-inetd
0 upgraded, 5 newly installed, 0 to remove and 45 not upgraded.
Need to get 125 kB of archives.
After this operation, 414 kB of additional disk space will be used.
0% [Working]

```

install : `sudo apt install vsftpd`

start server :

`sudo systemctl start vsftpd`

`sudo systemctl enable vsftpd`

and then install telnetd for open SSH

telnet install :

`sudo apt install telnetd -y`

then install openssh server

```
Processing triggers for ureadahead (0.100.0-21) ...
Processing triggers for systemd (237-3ubuntu10.57) ...
machineone@vulnmachine:/var/www/html/web-app/public$ sudo apt-get install openssh-server
```

then start and enable open ssh server

```
machineone@vulnmachine:/var/www/html/web-app/public$ sudo systemctl start ssh
machineone@vulnmachine:/var/www/html/web-app/public$ _
```

```
machineone@vulnmachine:/var/www/html/web-app/public$ sudo systemctl enable ssh --now
Synchronizing state of ssh.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
machineone@vulnmachine:/var/www/html/web-app/public$ _
```

we can connect open ssh by this process

you can find ip of your Ubuntu server by `ifconfig` command

```
machineone@vulmachine:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.242.128 netmask 255.255.255.0 broadcast 192.168.242.255
    inet6 fe80::20c:29ff:febc:41a prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:bc:04:1a txqueuelen 1000 (Ethernet)
    RX packets 137 bytes 12601 (12.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 107 bytes 13921 (13.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 84 bytes 6352 (6.3 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 84 bytes 6352 (6.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

machineone@vulmachine:~$ _
```

then in windows you can login by `machineone@192.168.242.128 {yourusername@ipaddress}`


```

C:\Users\dhruv>ssh machineone@192.168.242.128
machineone@192.168.242.128's password:
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-213-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Tue Mar 31 13:36:05 UTC 2026

System load:  0.0                       Processes:    198
Usage of /:   30.3% of 18.53GB          Users logged in: 1
Memory usage: 6%                       IP address for ens33: 192.168.242.128
Swap usage:   0%

41 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue Mar 31 13:33:16 2026
machineone@vulmachine:~$ |

```

You can access your machine remotely:

```

machineone@vulmachine:~$ ls -la
total 36
drwxr-xr-x 5 machineone machineone 4096 Mar 31 13:31 .
drwxr-xr-x 3 root        root        4096 Mar 31 12:55 ..
-rw-r----- 1 machineone machineone   37 Mar 31 13:33 .bash_history
-rw-r--r-- 1 machineone machineone  220 Apr  4 2018 .bash_logout
-rw-r--r-- 1 machineone machineone 3771 Apr  4 2018 .bashrc
drwx----- 2 machineone machineone 4096 Mar 31 12:56 .cache
drwx----- 3 machineone machineone 4096 Mar 31 12:56 .gnupg
-rw-r--r-- 1 machineone machineone  807 Apr  4 2018 .profile
drwx----- 2 machineone machineone 4096 Mar 31 12:55 .ssh
-rw-r--r-- 1 machineone machineone    0 Mar 31 13:05 .sudo_as_admin_successful
machineone@vulmachine:~$ |

```

Tools Used:

Zap

The ZAP Homepage

Welcome to ZAP!

 <https://www.zaproxy.org/>



```

(kali@kali)-[~]
$ sudo apt-get install zaproxy
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done

```

Burp Suite: already in the kali linux

Kali Linux / Packages / burpsuite · GitLab

burpsuite packaging for Kali Linux

 <https://gitlab.com/kalilinux/packages/burpsuite>



Nessus :

By-tenable

Install via curl

```
curl --request GET \
  --url 'https://www.tenable.com/downloads/api/v2/pages/nessus/files/Nessus-10.1
1.3-debian10_amd64.deb' \
  --output 'Nessus-10.11.3-debian10_amd64.deb'
```

```
(kali@kali)-[~/Downloads]
$ curl --request GET \
  --url 'https://www.tenable.com/downloads/api/v2/pages/nessus/files/Nessus-10.11.3-debian1
d64.deb' \
  --output 'Nessus-10.11.3-debian10_amd64.deb'
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           %             %             Dload  Upload  Total  Spent  Left   Speed
100 28.30M    0 28.30M    0     0  4.35M      0  0:06  0:06  0:06  5.63M
```

install deb package via `sudo dpkg -i {deb pkg name }`

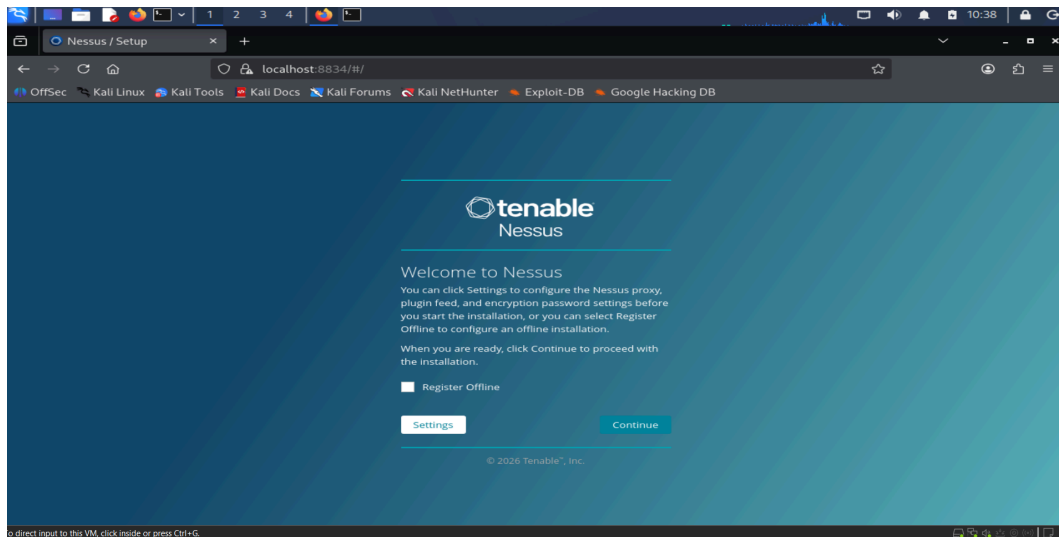
and then start Nessus

```
(kali@kali)-[~/Downloads]
$ sudo systemctl start nessusd.service
(kali@kali)-[~/Downloads]
$
```

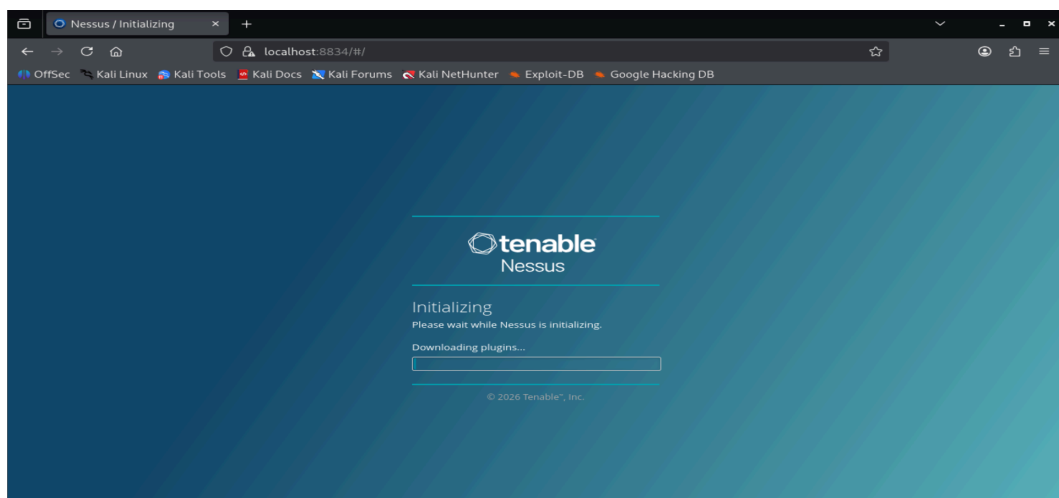
You can access Nessus via local host via port 8834

<https://localhost:8834/>

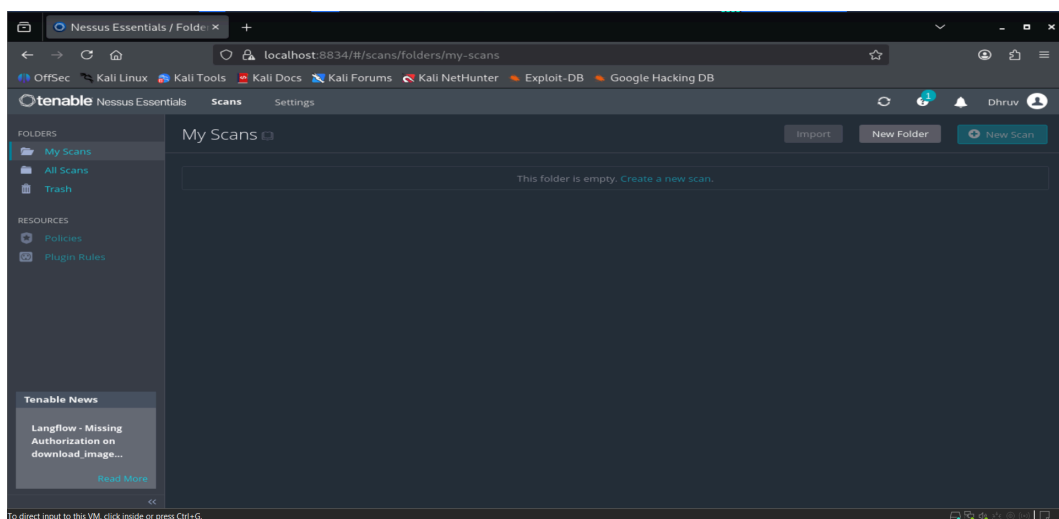
<https://localhost:8834/>



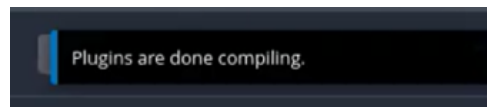
setup your account and then it'll download plugins



after setup the compiling will happen in the dashboard:



after compiling you'll get message



Nmap : pre-installed in Kali-linux

<https://github.com/nmap/nmap>

pwndoc : Pentesting report generator

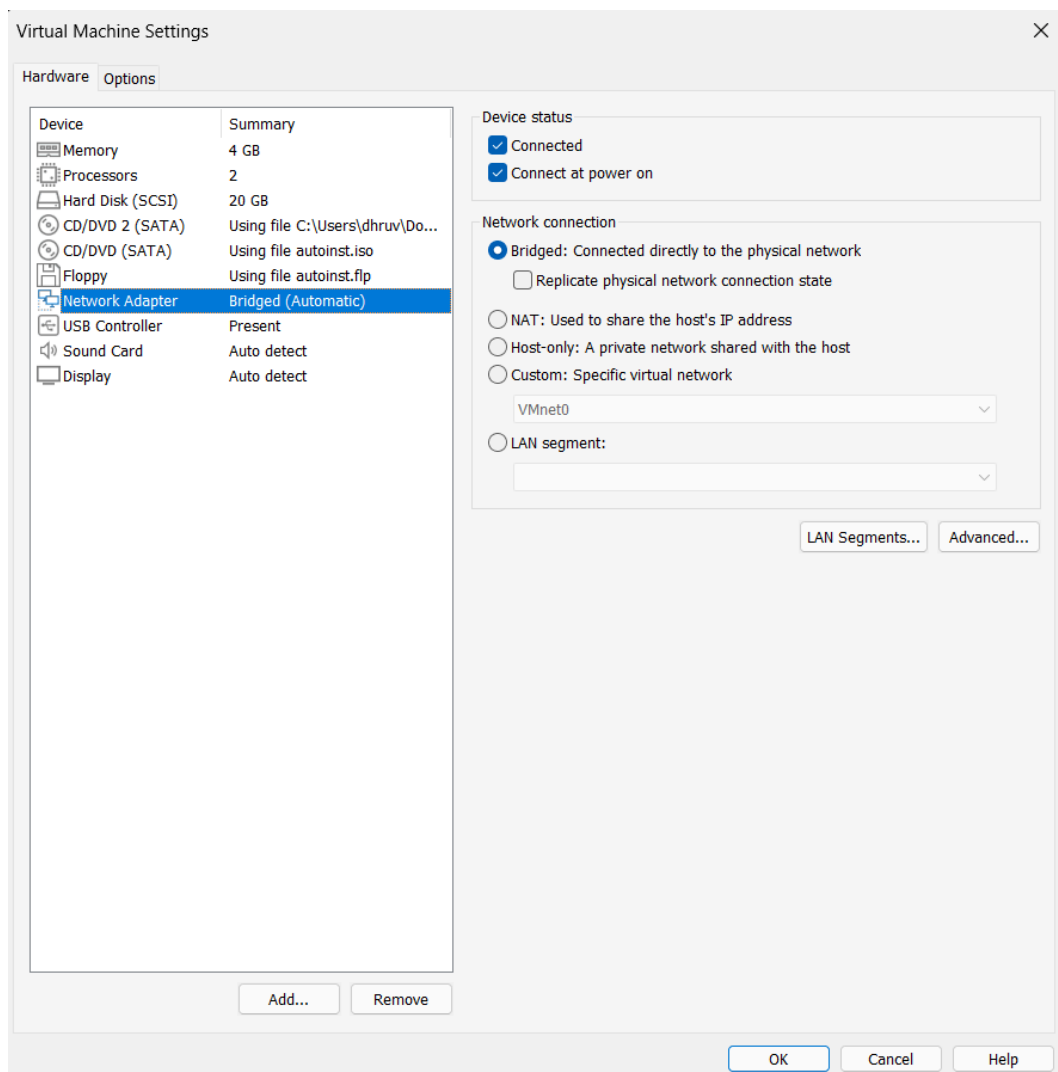
Document

Description

<https://pwndoc.github.io/pwndoc/#/>

ubuntu Server setup :

set network adapter to Bridge (automatic)



setup apache2 server via

```
machineone@vulnmachine:~$ sudo apt-get install apache2
```

configure server name:

```
machineone@vulnmachine:~$ sudo nano /etc/apache2/apache2.conf _
```

under #serverRoot section in nano text editor

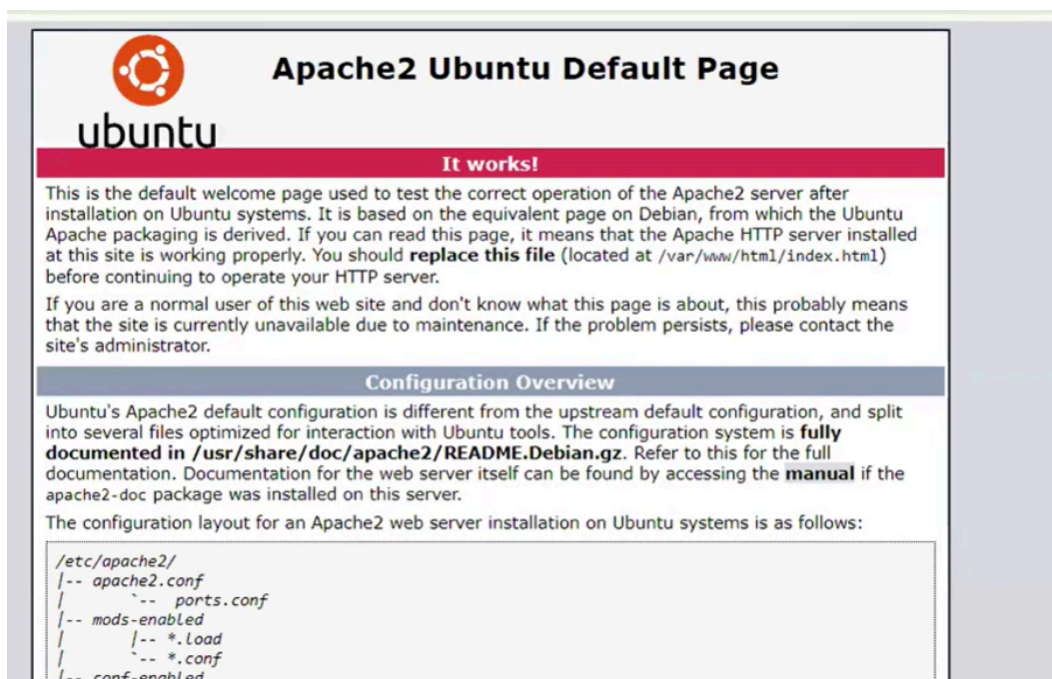
```
#ServerRoot "/etc/apache2"

ServerName 192.168.0.182
```

then you can expect this response from ubuntu server

```
machineone@vulnmachine:~$ sudo apache2ctl configtest
Syntax OK
machineone@vulnmachine:~$ sudo systemctl restart apache2
```

your server will be active or your machine ip



keep the firewall inactive :

```
machineone@vulnmachine:~$ sudo ufw status
Status: inactive
```

setup mysql server:

```
machineone@vulnmachine:~$ sudo apt-get install mysql-server
Reading package lists... Done
```

setup sqlserver:

```
machineone@vulnmachine:~$ mysql_secure_installation

Securing the MySQL server deployment.

Enter password for user root:
Error: Access denied for user 'root'@'localhost'
machineone@vulnmachine:~$ sudo mysql_secure_installation

Securing the MySQL server deployment.

Connecting to MySQL using a blank password.

VALIDATE PASSWORD PLUGIN can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD plugin?

Press y|Y for Yes, any other key for No: n
Please set the password for root here.
```

```
Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.
```

```
Disallow root login remotely? (Press y|Y for Yes, any other key for No) : n
```

```
Remove test database and access to it? (Press y|Y for Yes, any other key for No) : y
- Dropping test database...
Success.
```

setup PHP :

```
machineone@vulnmachine:~$ sudo apt-get install php libapache2-mod-php php-mysql
Reading package lists... Done
Building dependency tree
```

Install Git

```
machineone@vulnmachine:~$ sudo apt-get install git -y
Reading package lists... Done
Building dependency tree
```

Changes in Apache2

```
machineone@vulnmachine:~$ sudo nano /etc/apache2/mods-enabled/dir.conf_
```

reflect these changes as it is

```
GNU nano 2.9.3 /etc/apache2/mods-enabled/dir.conf
<IfModule mod_dir.c>
    DirectoryIndex index.php index.html index.cgi index.pl index.xhtml index.htm
</IfModule>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
```

restart apache2

```
machineone@vulnmachine:~$ sudo systemctl restart apache2
```

you can check apache2 server status by

```
machineone@vulnmachine:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
  Drop-In: /lib/systemd/system/apache2.service.d
           └─apache2-systemd.conf
   Active: active (running) since Mon 2024-09-30 14:05:47 UTC; 10s ago
     Process: 11379 ExecStop=/usr/sbin/apachectl stop (code=exited, status=0/SUCCESS)
     Process: 11384 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 11401 (apache2)
      Tasks: 6 (limit: 4623)
   CGroup: /system.slice/apache2.service
           └─11401 /usr/sbin/apache2 -k start
             11405 /usr/sbin/apache2 -k start
             11407 /usr/sbin/apache2 -k start
             11409 /usr/sbin/apache2 -k start
             11411 /usr/sbin/apache2 -k start
             11413 /usr/sbin/apache2 -k start

Sep 30 14:05:47 vulnmachine systemd[1]: Starting The Apache HTTP Server...
Sep 30 14:05:47 vulnmachine systemd[1]: Started The Apache HTTP Server.
machineone@vulnmachine:~$
```

some more php setup

```
machineone@vulnmachine:~$ sudo apt-cache search php- | less
```

we will install all dev packages for php

```
machineone@vulnmachine:~$ sudo apt-get install -y php-all-dev
```

Setting up a file to check php is running or not

```
machineone@vulnmachine:~$ sudo nano /var/www/html/info.php
```

```

GNU nano 2.9.3 /var/www/html/info.php

<?php
phpinfo();
?>

```

you can get your php info on :
ipaddress/info.php

Not secure
ipaddress/info.php

PHP Version 7.2.24-0ubuntu0.18.04.17


System	Linux vulnmachine 4.15.0-213-generic #224-Ubuntu SMP Mon Jun 19 13:30:12 UTC 2023 x86_64
Build Date	Feb 23 2023 13:29:25
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.2/apache2
Loaded Configuration File	/etc/php/7.2/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.2/apache2/conf.d
Additional .ini files parsed	/etc/php/7.2/apache2/conf.d/10-mysqld.ini, /etc/php/7.2/apache2/conf.d/10-opcache.ini, /etc/php/7.2/apache2/conf.d/10-pdo.ini, /etc/php/7.2/apache2/conf.d/20-calendar.ini, /etc/php/7.2/apache2/conf.d/20-ctype.ini, /etc/php/7.2/apache2/conf.d/20-exif.ini, /etc/php/7.2/apache2/conf.d/20-fileinfo.ini, /etc/php/7.2/apache2/conf.d/20-ftp.ini, /etc/php/7.2/apache2/conf.d/20-gd.ini, /etc/php/7.2/apache2/conf.d/20-gettext.ini, /etc/php/7.2/apache2/conf.d/20-iconv.ini, /etc/php/7.2/apache2/conf.d/20-json.ini, /etc/php/7.2/apache2/conf.d/20-mysqli.ini, /etc/php/7.2/apache2/conf.d/20-pdo_mysql.ini, /etc/php/7.2/apache2/conf.d/20-phar.ini, /etc/php/7.2/apache2/conf.d/20-posix.ini, /etc/php/7.2/apache2/conf.d/20-readline.ini, /etc/php/7.2/apache2/conf.d/20-shmop.ini, /etc/php/7.2/apache2/conf.d/20-sockets.ini, /etc/php/7.2/apache2/conf.d/20-sysmsg.ini, /etc/php/7.2/apache2/conf.d/20-sysvsem.ini, /etc/php/7.2/apache2/conf.d/20-sysvshm.ini, /etc/php/7.2/apache2/conf.d/20-tokenizer.ini
PHP API	20170718
PHP Extension	20170718
Zend Extension	320170718
Zend Extension Build	API320170718.NTS
PHP Extension Build	API20170718.NTS
Debug Build	no
Thread Safety	disabled

setup composer:

```

machineone@vulnmachine:~$ sudo apt-get install composer -y
Reading package lists... Done

machineone@vulnmachine:~$ composer -v_

```

Redirection problem fix :

```

machineone@vulnmachine:~$ cd /var/www/html/web-app/public
machineone@vulnmachine:/var/www/html/web-app/public$ ls -la
total 40
drwxrwsr-x 5 machineone machineone 4096 Sep 30 16:45 .
drwxrwsr-x 14 machineone machineone 4096 Sep 30 16:57 ..
drwxrwsr-x 2 machineone machineone 4096 Sep 30 16:45 css
-rw-rw-r-- 1 machineone machineone 0 Sep 30 16:45 favicon.ico
-rw-rw-r-- 1 machineone machineone 593 Sep 30 16:45 htaccess
-rw-rw-r-- 1 machineone machineone 1823 Sep 30 16:45 index.php
drwxrwsr-x 2 machineone machineone 4096 Sep 30 16:45 js
-rw-rw-r-- 1 machineone machineone 71 Sep 30 16:45 mix-manifest.json
-rw-rw-r-- 1 machineone machineone 24 Sep 30 16:45 robots.txt
-rw-rw-r-- 1 machineone machineone 66 Sep 30 16:45 test.php
drwxrwsr-x 2 machineone machineone 4096 Sep 30 16:45 uploads
machineone@vulnmachine:/var/www/html/web-app/public$ nano .htaccess _

```



```

<IfModule mod_rewrite.c>
    <IfModule mod_negotiation.c>
        Options -MultiViews -Indexes
    </IfModule>

    RewriteEngine On
    # Options +FollowSymLinks

    RewriteBase /
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule ^(.*)$ index.php/$1 [L]

    # Handle Authorization Header
    # RewriteCond %{HTTP:Authorization} .
    # RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]

    # Redirect Trailing Slashes If Not A Folder...
    # RewriteCond %{REQUEST_FILENAME} !-d
    # RewriteCond %{REQUEST_URI} (.+)/$
    # RewriteRule ^ %1 [L,R=301]

    # Handle Front Controller...
    # RewriteCond %{REQUEST_FILENAME} !-d
    # RewriteCond %{REQUEST_FILENAME} !-f
    # RewriteRule ^$ index.php [L]
</IfModule>

```

then restart apache2

```
machineone@vulnmachine:/var/www/html/web-app/public$ sudo systemctl restart apache2
```

Change these as well in this directory

```
machineone@vulnmachine:/etc/apache2$ sudo nano sites-enabled/000-default.conf
[sudo] password for machineone:
```

```

GNU nano 2.9.3      sites-enabled/000-default.conf      Modified

<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/web-app/public

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
    <Directory "/var/www/html/web-app/public">
        AllowOverride All
    </Directory>
    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>

```

changes Htaccess

```

machineone@vulnmachine:/etc/apache2$ cd /var/www/html/web-app/
machineone@vulnmachine:/var/www/html/web-app$ ls
app          composer.lock  docker-compose.yml  public      server.php      test.txt
artisan      config         Dockerfile          README.md   storage         vendor
bootstrap   database       package.json        resources   tailwind.config.js  webpack.mix.js
composer.json  docker         phpunit.xml         routes      tests           yarn.lock
machineone@vulnmachine:/var/www/html/web-app$ cd public/
machineone@vulnmachine:/var/www/html/web-app/public$ ls
css  favicon.ico  index.php  js  mix-manifest.json  robots.txt  test.php  uploads
machineone@vulnmachine:/var/www/html/web-app/public$ nano .htaccess _

```

```

GNU nano 2.9.3 .htaccess

<IfModule mod_rewrite.c>
  <IfModule mod_negotiation.c>
    Options -MultiViews -Indexes
  </IfModule>

  RewriteEngine On
  # Options +FollowSymLinks

  RewriteBase /
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteRule ^(.*)$ index.php/$1 [L]

  # Handle Authorization Header
  # RewriteCond %{HTTP:Authorization} .
  # RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]

  # Redirect Trailing Slashes If Not A Folder...
  # RewriteCond %{REQUEST_FILENAME} !-d
  # RewriteCond %{REQUEST_URI} (.+)/$
  # RewriteRule ^%1 [L,R=301]

  # Handle Front Controller...
  # RewriteCond %{REQUEST_FILENAME} !-d
  # RewriteCond %{REQUEST_FILENAME} !-f
  # RewriteRule ^$ index.php [L]
</IfModule>

```

Restart apache2 server

Good to go in Demo app webserver

Ip/register

Vulnerable PHP Demo App

Login
Register

Register

Name:

E-Mail Address:

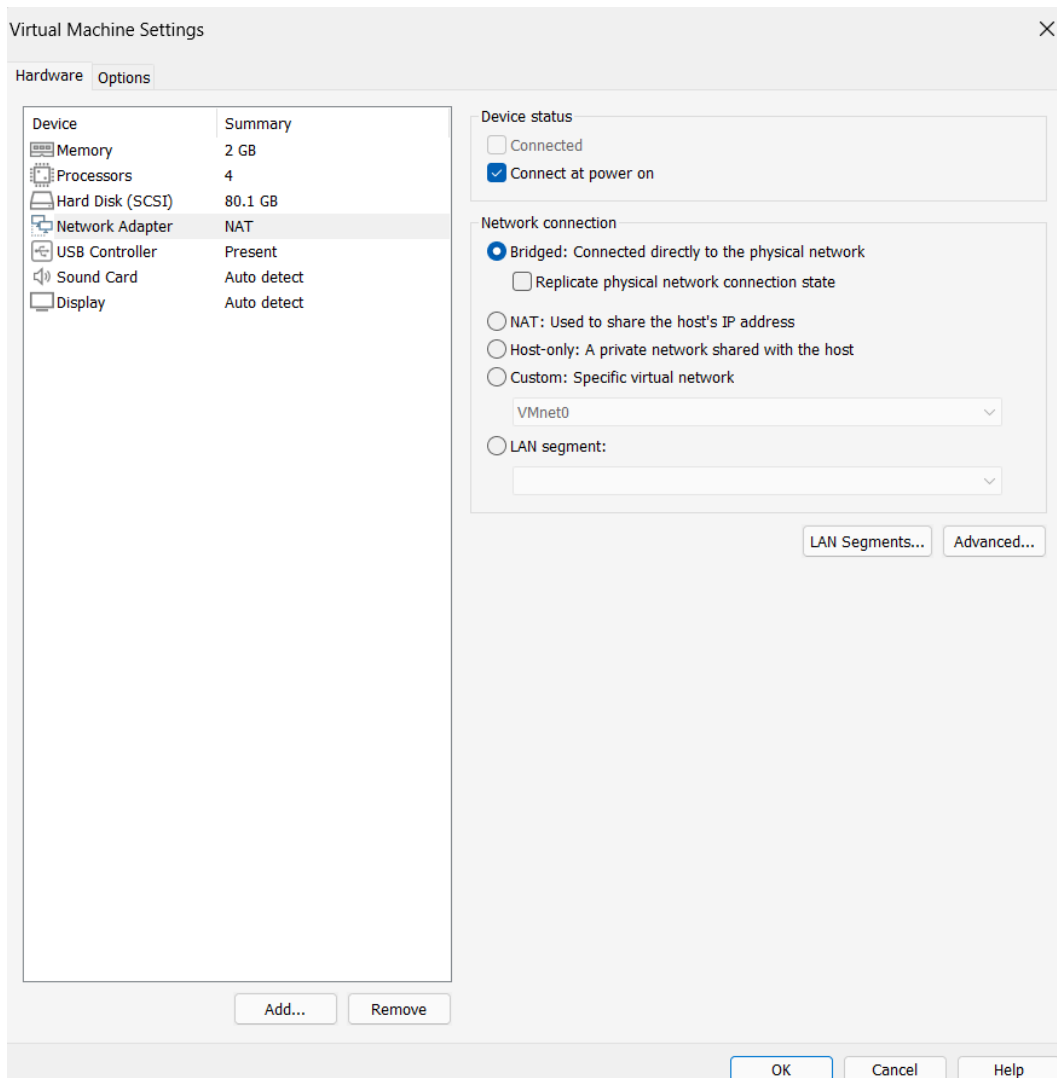
Password:

Confirm Password:

Register

Already have an account? Login

Change Kali machine Network adapter to Bridge as well to ensure that are on same network



Check your Ip via `ifconfig`

Run Nmap Scan on your local Machine

```

└─$ nmap 192.168.0.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-09-30 14:18 EDT
Nmap scan report for dlinkrouter (192.168.0.1)
Host is up (0.015s latency).
Not shown: 993 closed tcp ports (conn-refused)
PORT      STATE SERVICE
53/tcp    open  domain

```

vulnerable laravel app setup

<https://github.com/dennisehcs/vulnerable-laravel-app>

Permission fix

```

machineone@vulnmachine:/var$ groups
machineone adm cdrom sudo dip plugdev lxd
machineone@vulnmachine:/var$ sudo chown -R $USER:$USER /var/www
[sudo] password for machineone:

```

Run these commands to give full permission acces to yourself

```
machineone@vulnmachine:/var$ ls -la
total 56
drwxr-xr-x 14 root    root    4096 Sep 30 13:54 .
drwxr-xr-x 24 root    root    4096 Sep 30 13:21 ..
drwxr-xr-x  2 root    root    4096 Apr 24 2018 backups
drwxr-xr-x 10 root    root    4096 Sep 30 13:54 cache
drwxrwxrwt  2 root    root    4096 Sep 15 2021 crash
drwxr-xr-x 45 root    root    4096 Sep 30 14:03 lib
drwxrwsr-x  2 root    staff   4096 Apr 24 2018 local
lrwxrwxrwx  1 root    root      9 Sep 15 2021 lock -> /run/lock
drwxrwxr-x 11 root    syslog  4096 Sep 30 16:35 log
drwxrwsr-x  2 root    mail    4096 Sep 15 2021 mail
drwxr-xr-x  2 root    root    4096 Sep 15 2021 opt
lrwxrwxrwx  1 root    root      4 Sep 15 2021 run -> /run
drwxr-xr-x  2 root    root    4096 Mar 26 2021 snap
drwxr-xr-x  4 root    root    4096 Sep 15 2021 spool
drwxrwxrwt  5 root    root    4096 Sep 30 16:39 tmp
drwxr-xr-x  3 machineone machineone 4096 Sep 30 13:54 www
machineone@vulnmachine:/var$ cd www/
machineone@vulnmachine:/var/www$ ls
html
machineone@vulnmachine:/var/www$ ls -la
total 12
drwxr-xr-x  3 machineone machineone 4096 Sep 30 13:54 .
drwxr-xr-x 14 root        root        4096 Sep 30 13:54 ..
drwxr-xr-x  2 machineone machineone 4096 Sep 30 14:10 html
machineone@vulnmachine:/var/www$
```

```
machineone@vulnmachine:/var/www$ sudo chmod 2777 /var/www && find /var/www -type d -exec sudo chmod 2777 {} \;
machineone@vulnmachine:/var/www$ ls -la
total 12
drwxrwsrwx  3 machineone machineone 4096 Sep 30 13:54 .
drwxr-xr-x 14 root        root        4096 Sep 30 13:54 ..
drwxrwsrwx  2 machineone machineone 4096 Sep 30 14:10 html
machineone@vulnmachine:/var/www$
```

```
machineone@vulnmachine:/var/www$ find /var/www -type f -exec sudo chmod 0777 {} \;
machineone@vulnmachine:/var/www$ ls -la
total 12
drwxrwsrwx  3 machineone machineone 4096 Sep 30 13:54 .
drwxr-xr-x 14 root        root        4096 Sep 30 13:54 ..
drwxrwsrwx  2 machineone machineone 4096 Sep 30 14:10 html
machineone@vulnmachine:/var/www$ cd html/
machineone@vulnmachine:/var/www/html$ ls
index.html  info.php
machineone@vulnmachine:/var/www/html$ ls -la
total 24
drwxrwsrwx 2 machineone machineone 4096 Sep 30 14:10 .
drwxrwsrwx 3 machineone machineone 4096 Sep 30 13:54 ..
-rwxrwxrwx 1 machineone machineone 10918 Sep 30 13:54 index.html
-rwxrwxrwx 1 machineone machineone 20 Sep 30 14:10 info.php
machineone@vulnmachine:/var/www/html$
```

Now clone git hub directory to folder <https://github.com/dennisehcs/vulnerable-laravel-app>

```
machineone@vulnmachine:/var/www/html$ git clone https://github.com/dennisehcs/vulnerable-laravel-app
Cloning into 'vulnerable-laravel-app'...
remote: Enumerating objects: 255, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 255 (delta 13), reused 12 (delta 12), pack-reused 237 (from 1)
Receiving objects: 100% (255/255), 588.66 KiB | 2.75 MiB/s, done.
Resolving deltas: 100% (77/77), done.
machineone@vulnmachine:/var/www/html$
```

rename to web-app:

```
machineone@vulnmachine:/var/www/html$ mv vulnerable-laravel-app/ web-app
machineone@vulnmachine:/var/www/html$ ls
```

log error

```

machineone@vulnmachine:/var/www/html/web-app$ composer install 2>> install-output.log
machineone@vulnmachine:/var/www/html/web-app$ ls
app          config      install-output.log  README.md  tailwind.config.js
artisan     database   install-output.txt  resources  tests
bootstrap  docker     package.json        routes     test.txt
composer.json  docker-compose.yml  phpunit.xml        server.php  webpack.mix.js
composer.lock Dockerfile  public              storage     yarn.lock
machineone@vulnmachine:/var/www/html/web-app$ _

```

install mbstring

```

machineone@vulnmachine:/var/www/html/web-app$ sudo apt-get install php-mbstring
[sudo] password for machineone:

```

```

machineone@vulnmachine:/var/www/html/web-app$ sudo apt-get install php-zip -y
[sudo] password for machineone:
Reading package lists... Done

```

now install composer

```

machineone@vulnmachine:/var/www/html/web-app$ composer install
Loading composer repositories with package information
Installing dependencies (including require-dev) from lock file
Package operations: 74 installs, 0 updates, 0 removals
 - Installing symfony/var-dumper (v4.3.3): Downloading (connecting...)_

```

ip/web-app is accessible so that is a Vulnerability

Index of /web-app

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 Dockerfile	2024-09-30 16:45	1.2K	
 README.md	2024-09-30 16:45	1.3K	
 app/	2024-09-30 16:45	-	
 artisan	2024-09-30 16:45	1.6K	
 bootstrap/	2024-09-30 16:45	-	
 composer.json	2024-09-30 16:45	1.7K	
 composer.lock	2024-09-30 16:45	173K	
 config/	2024-09-30 16:45	-	
 database/	2024-09-30 16:45	-	
 docker-compose.yml	2024-09-30 16:45	708	
 docker/	2024-09-30 16:45	-	
 package.json	2024-09-30 16:45	1.2K	
 phpunit.xml	2024-09-30 16:45	1.1K	
 public/	2024-09-30 16:45	-	
 resources/	2024-09-30 16:45	-	
 routes/	2024-09-30 16:45	-	
 server.php	2024-09-30 16:45	563	
 storage/	2024-09-30 16:45	-	

open sql:

```
machineone@vulnmachine:/var/www/html/web-app/public$ sudo mysql
[sudo] password for machineone:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2
Server version: 5.7.42-0ubuntu0.18.04.1 (Ubuntu)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```


create database:

```
mysql> CREATE DATABASE vulndb
-> ;
Query OK, 1 row affected (0.00 sec)
```

check database:

```
mysql> show databases
-> ;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| vulndb |
+-----+
5 rows in set (0.01 sec)
```

Look for .env file

```
machineone@vulnmachine:/var/www/html/web-app$ ls -la
total 588
drwxrwsr-x 14 machineone machineone 4096 Sep 30 16:53 .
drwxrwsr-x  3 machineone machineone 4096 Sep 30 16:46 ..
drwxrwsr-x  6 machineone machineone 4096 Sep 30 16:45 app
-rw-rw-r--  1 machineone machineone 1686 Sep 30 16:45 artisan
drwxrwsr-x  3 machineone machineone 4096 Sep 30 16:45 bootstrap
-rw-rw-r--  1 machineone machineone 1715 Sep 30 16:45 composer.json
-rw-rw-r--  1 machineone machineone 176852 Sep 30 16:45 composer.lock
drwxrwsr-x  2 machineone machineone 4096 Sep 30 16:45 config
drwxrwsr-x  5 machineone machineone 4096 Sep 30 16:45 database
drwxrwsr-x  3 machineone machineone 4096 Sep 30 16:45 docker
-rw-rw-r--  1 machineone machineone 708 Sep 30 16:45 docker-compose.yml
-rw-rw-r--  1 machineone machineone 1214 Sep 30 16:45 Dockerfile
-rw-rw-r--  1 machineone machineone 213 Sep 30 16:45 .editorconfig
-rw-rw-r--  1 machineone machineone 726 Sep 30 16:45 .env.example
drwxrwsr-x  8 machineone machineone 4096 Sep 30 16:45 .git
-rw-rw-r--  1 machineone machineone 111 Sep 30 16:45 .gitattributes
-rw-rw-r--  1 machineone machineone 188 Sep 30 16:45 .gitignore
-rw-rw-r--  1 machineone machineone 1196 Sep 30 16:45 package.json
-rw-rw-r--  1 machineone machineone 1156 Sep 30 16:45 phpunit.xml
drwxrwsr-x  5 machineone machineone 4096 Sep 30 16:45 public
-rw-rw-r--  1 machineone machineone 1294 Sep 30 16:45 README.md
drwxrwsr-x  6 machineone machineone 4096 Sep 30 16:45 resources
drwxrwsr-x  2 machineone machineone 4096 Sep 30 16:45 routes
-rw-rw-r--  1 machineone machineone 563 Sep 30 16:45 server.php
drwxrwsr-x  5 machineone machineone 4096 Sep 30 16:45 storage
-rw-rw-r--  1 machineone machineone 174 Sep 30 16:45 .styleci.yml
-rw-rw-r--  1 machineone machineone 276 Sep 30 16:45 tailwind.config.js
drwxrwsr-x  4 machineone machineone 4096 Sep 30 16:45 tests
-rw-rw-r--  1 machineone machineone 9 Sep 30 16:45 test.txt
drwxrwsr-x 42 machineone machineone 4096 Sep 30 16:53 vendor
-rw-rw-r--  1 machineone machineone 703 Sep 30 16:45 webpack.mix.js
-rw-rw-r--  1 machineone machineone 298278 Sep 30 16:45 yarn.lock
machineone@vulnmachine:/var/www/html/web-app$ _
```

copy it

```
machineone@vulnmachine:/var/www/html/web-app$ cp .env.example .env
```

generate key

```
machineone@vulnmachine:/var/www/html/web-app$ php artisan key:generate
Application key set successfully.
machineone@vulnmachine:/var/www/html/web-app$
```

open .env file do some changes

```
machineone@vulnmachine:/var/www/html/web-app$ nano .env
```

Change DB name and Password

```
DB_PORT=3306
DB_DATABASE=vulndb
DB_USERNAME=root
DB_PASSWORD=1234
```

Vulnerabilities scan

Nmap Scan

Network scan 1


```
(kali@kali)-[~]
$ nmap 192.168.0.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-09-30 14:30 EDT
Nmap scan report for dlinkrouter (192.168.0.1)
Host is up (0.062s latency).
Not shown: 993 closed tcp ports (conn-refused)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
81/tcp    open  hosts2-ns
443/tcp   open  https
4445/tcp  open  upnotifyp
8888/tcp  open  sun-answerbook
9999/tcp  open  abyss

Nmap scan report for DESKTOP-K3C87A8 (192.168.0.115)
Host is up (0.00037s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp   open  https
3306/tcp  open  mysql

Nmap scan report for 192.168.0.116
Host is up (0.0077s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
62078/tcp open  iphone-sync

Nmap scan report for kali (192.168.0.180)
Host is up (0.000039s latency).
All 1000 scanned ports on kali (192.168.0.180) are in ignored states.
Not shown: 1000 closed tcp ports (conn-refused)

Nmap scan report for vulnmachine (192.168.0.182)
Host is up (0.00079s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http

Nmap done: 256 IP addresses (5 hosts up) scanned in 20.85 seconds
```

Network scan 2

```
(kali@kali)-[~]
$ nmap -sn 192.168.0.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-09-30 14:31 EDT
Nmap scan report for dlinkrouter (192.168.0.1)
Host is up (0.0013s latency).
Nmap scan report for Nokia-C21-Plus (192.168.0.111)
Host is up (0.061s latency).
Nmap scan report for DESKTOP-K3C87A8 (192.168.0.115)
Host is up (0.00049s latency).
Nmap scan report for 192.168.0.116
Host is up (0.0056s latency).
Nmap scan report for kali (192.168.0.180)
Host is up (0.00031s latency).
Nmap scan report for vulnmachine (192.168.0.182)
Host is up (0.00031s latency).
Nmap done: 256 IP addresses (6 hosts up) scanned in 2.33 seconds
```

Network scan 3

```

(kali@kali)-[~]
$ nmap -A -T4 192.168.0.182
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-09-30 14:32 EDT
Nmap scan report for vulnmachine (192.168.0.182)
Host is up (0.00094s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 44:af:7a:31:1b:4b:76:79:d8:36:f6:79:02:e1:c5:f1 (RSA)
|   256 7d:d3:16:51:de:52:0b:17:d3:0d:1d:a3:13:f3:25:4b (ECDSA)
|_  256 3a:58:93:c6:8f:c5:b6:24:d8:d8:7c:26:3a:1f:fe:2e (ED25519)
23/tcp    open  telnet   Linux telnetd
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: Laravel
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.54 seconds

```

Network scan 3

```

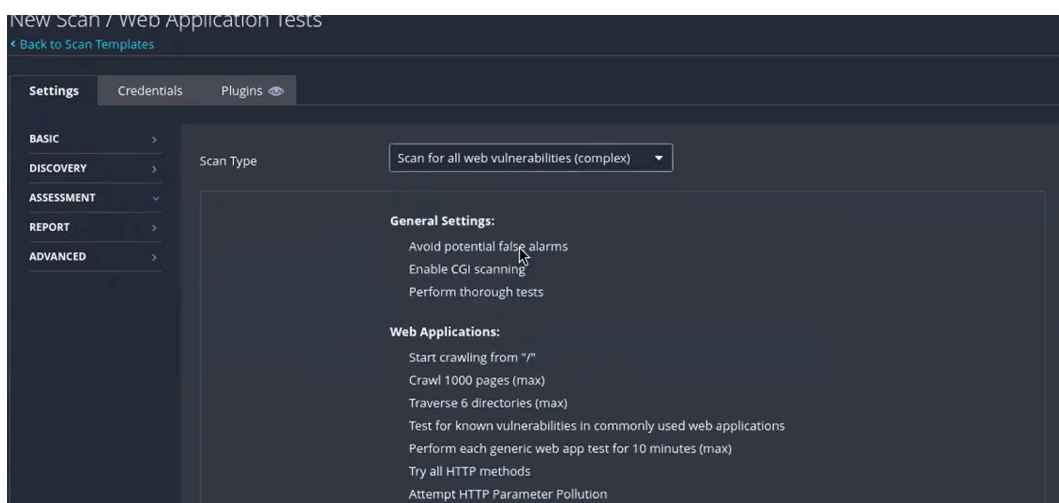
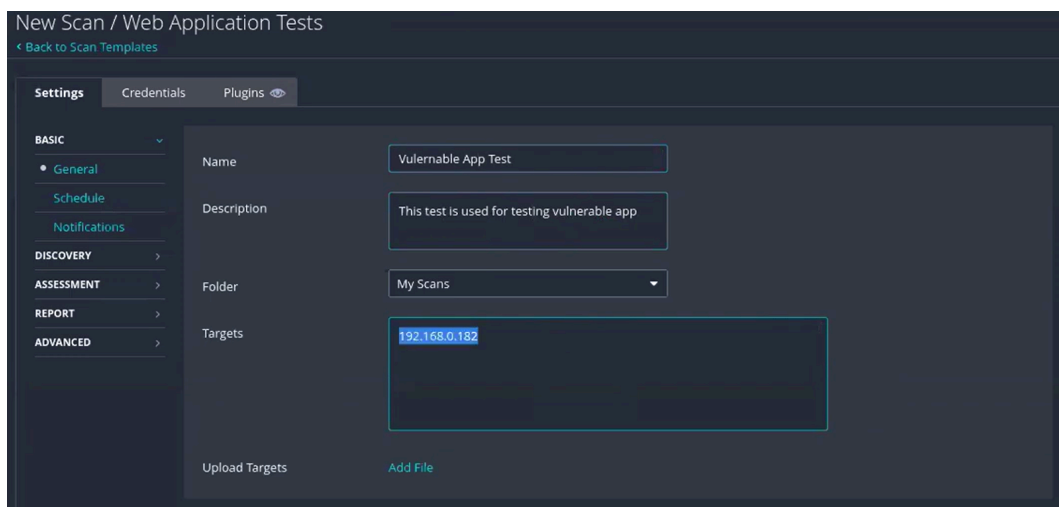
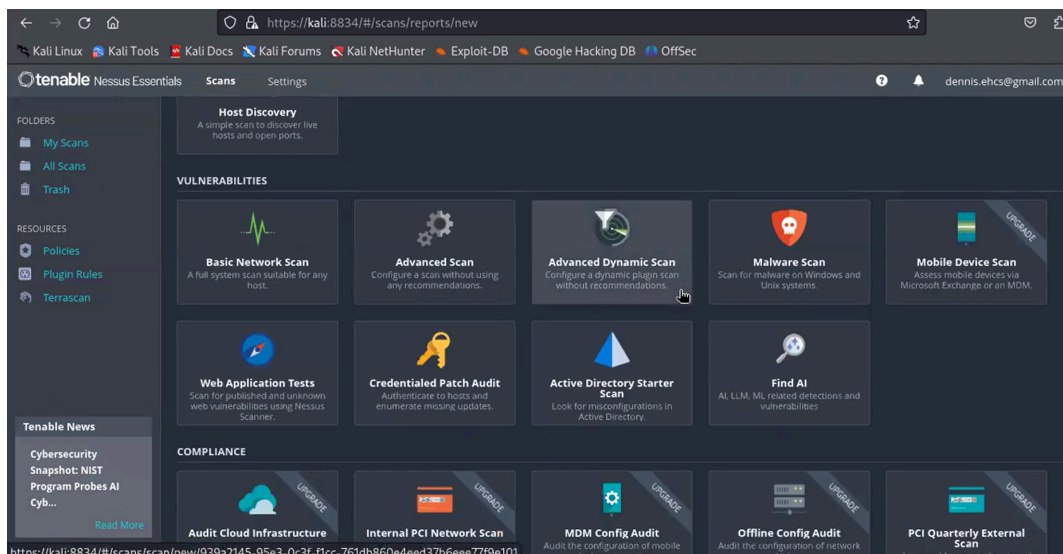
(kali@kali)-[~]
$ sudo nmap -Pn --script vuln 192.168.0.182
[sudo] password for kali:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-09-30 14:33 EDT
Nmap scan report for vulnmachine (192.168.0.182)
Host is up (0.00012s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
|_ http-dombased-xss: Couldn't find any DOM based XSS.
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_ http-csrf: Couldn't find any CSRF vulnerabilities.
| http-enum:
|   /login/: Login page
|   /test.php: Test page
|   /robots.txt: Robots file
|_  /register/: Potentially interesting folder
MAC Address: 00:0C:29:52:3B:42 (VMware)

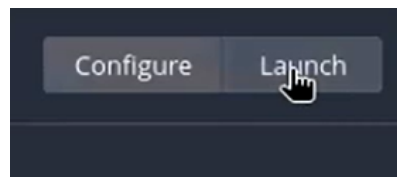
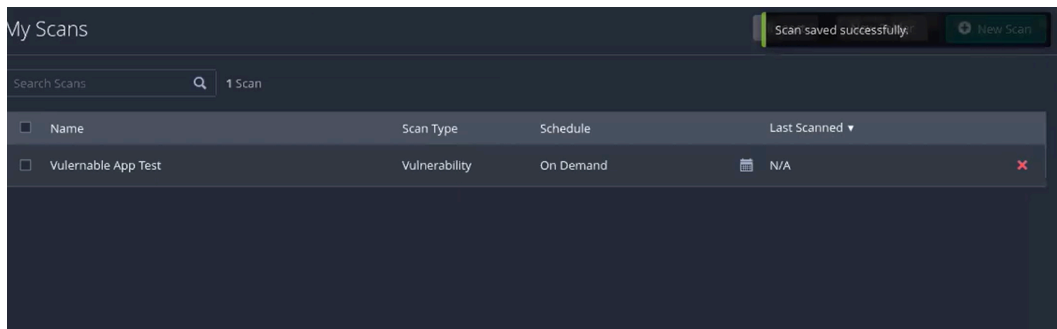
Nmap done: 1 IP address (1 host up) scanned in 50.00 seconds

(kali@kali)-[~]
$ 

```

Nessus





Some Nessus Vulnerabilities :

MEDIUM
Web Application Potentially Vulnerable to Clickjacking

Description
The remote web server does not set an X-Frame-Options response header or a Content-Security-Policy 'frame-ancestors' response header in all content responses. This could potentially expose the site to a clickjacking or UI redress attack, in which an attacker can trick a user into clicking an area of the vulnerable page that is different than what the user perceives the page to be. This can result in a user performing fraudulent or malicious transactions.

X-Frame-Options has been proposed by Microsoft as a way to mitigate clickjacking attacks and is currently supported by all major browser vendors.

Content-Security-Policy (CSP) has been proposed by the W3C Web Application Security Working Group, with increasing support among all major browser vendors, as a way to mitigate clickjacking and other attacks. The 'frame-ancestors' policy directive restricts which sources can embed the protected resource.

Note that while the X-Frame-Options and Content-Security-Policy response headers are not the only mitigations for clickjacking, they are currently the most reliable methods that can be detected through automation. Therefore, this plugin may produce false positives if other mitigation strategies (e.g., frame-busting JavaScript) are deployed or if the page does not perform any security-sensitive transactions.

Vulnerability: Web Application Potentially Vulnerable to Clickjacking

The Core Issue

The web application does not restrict how its pages can be framed by other websites. Because the server is missing specific security headers (`X-Frame-Options` or `Content-Security-Policy`), a malicious site can load your application inside an invisible `<iframe>` .

Real-World Scenario

An attacker creates a fake website offering a prize, like a "Claim Your Free Gift" page. They load your vulnerable web application in an invisible frame directly over their own webpage. They align a button on your application—such as "Transfer Funds", "Delete Account", or "Change Password"—exactly under the visible "Claim Gift" button.

When a user (who is already logged into your application) visits the attacker's site and clicks "Claim Gift," their click passes through the invisible layer and hits the button on your application instead. The user unknowingly performs a malicious action while thinking they are just clicking a harmless button.

Remediation Steps

To fix this, you need to configure your web server to send HTTP response headers that tell the browser whether the site is allowed to be rendered within a frame.

Implement one (or both) of the following security headers across all application pages:

- **Content-Security-Policy (CSP):** This is the modern standard. Add the `frame-ancestors` directive to your CSP header.
 - To block all framing: `Content-Security-Policy: frame-ancestors 'none';`
 - To allow framing only by pages on your own domain: `Content-Security-Policy: frame-ancestors 'self';`
- **X-Frame-Options (XFO):** This is an older header but still heavily used for backwards compatibility with older browsers.
 - To block all framing: `X-Frame-Options: DENY`
 - To allow framing only by pages on your own domain: `X-Frame-Options: SAMEORIGIN`

Return the X-Frame-Options or Content-Security-Policy (with the 'frame-ancestors' directive) HTTP header with the page's response. This prevents the page's content from being rendered by another site when using the frame or iframe HTML tags.

See Also

<http://www.nessus.org/u?399b1f56>
https://www.owasp.org/index.php/Clickjacking_Defense_Cheat_Sheet
<https://en.wikipedia.org/wiki/Clickjacking>

Output

The following pages do not use a clickjacking mitigation response header and contain a clickable event :

- `http://vulnmachine/login`
- `http://vulnmachine/password/reset`
- `http://vulnmachine/register`

To see debug logs, please visit individual host

Port ▲	Hosts
80 / tcp / www	192.168.0.182

LOW Web Server Transmits Cleartext Credentials

Description

The remote web server contains several HTML form fields containing an input of type 'password' which transmit their information to a remote web server in cleartext.

An attacker eavesdropping the traffic between web browser and server may obtain logins and passwords of valid users.

Solution

Make sure that every sensitive form transmits content over HTTPS.

Output

Page : /login
Destination Page: /login

Page : /register
Destination Page: /register

To see debug logs, please visit individual host

Port ▲	Hosts
80 / tcp / www	192.168.0.182

Vulnerability: Web Server Transmits Cleartext Credentials

The Core Issue

The web application is transmitting sensitive data—specifically passwords from the login and registration pages—over an unencrypted HTTP connection. Because the connection lacks SSL/TLS encryption, the data travels across the network in plain, human-readable text.

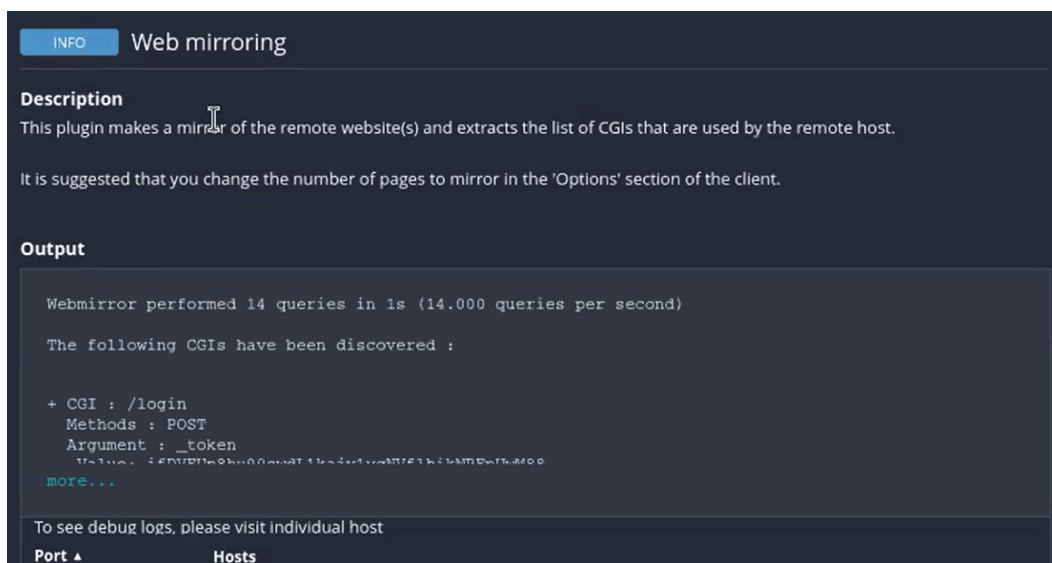
Real-World Scenario

A user logs into the application while connected to a public Wi-Fi network at a coffee shop or hotel. An attacker connected to the same network uses a standard packet sniffing tool (like Wireshark) to passively capture network traffic. Because the credentials are sent in cleartext over port 80, the attacker can instantly read the user's exact username and password from the captured traffic and use them to take over the account.

Remediation Steps

To fix this, you must encrypt the communication channel between the user's browser and the web server so that intercepted data cannot be read.

- **Enable HTTPS:** Install a valid SSL/TLS certificate on the web server and configure it to accept secure connections.
- **Force Redirection:** Configure the web server to automatically redirect all plain HTTP traffic (port 80) to secure HTTPS traffic (port 443).
- **Update Form Actions:** Verify that the HTML `<form>` tags on the affected pages (`/login` and `/register`) are explicitly configured to submit data to `https://` URLs.
- **Implement HSTS:** Add the HTTP Strict Transport Security (HSTS) response header. This instructs modern browsers to only interact with the application using secure HTTPS connections in the future, even if a user types `http://` in the address bar.



Finding: Web Mirroring (Informational)

The Core Issue

This is not actually a vulnerability; it is an informational finding. It indicates that the Nessus scanner successfully crawled (or "spidered") your web application to map out its structure. During this process, it identified interactive pages, forms, and scripts—such as the `/login` page and its associated input fields—to understand the application's attack surface.

Real-World Scenario

Before launching a targeted attack, a hacker will perform reconnaissance by running automated crawling tools against your website. Their goal is to map out the entire architecture to discover every possible entry point. They are specifically looking for hidden administrative portals, forgotten backup files, or input forms that they can later target with brute-force login attempts or SQL injection attacks.

Remediation Steps

Because this is an informational reconnaissance finding rather than a software flaw, there is no patch to apply. However, you should take the following defensive measures:

- **Review the Output:** Carefully check the list of pages and directories the scanner discovered. Ensure that no sensitive internal documents, test pages, or administrative interfaces are unintentionally exposed to the public internet.
- **Implement a WAF / Rate Limiting:** Configure a Web Application Firewall (WAF) or set up rate limiting on your web server. This helps detect and block aggressive automated tools and malicious bots from scraping your site.
- **Clean Up Orphaned Content:** Remove any deprecated or unused scripts, forms, and pages from the web server so they cannot be discovered and exploited later.

LOW Web Server Transmits Cleartext Credentials

Description

The remote web server contains several HTML form fields containing an input of type 'password' which transmit their information to a remote web server in cleartext.

An attacker eavesdropping the traffic between web browser and server may obtain logins and passwords of valid users.

Solution

Make sure that every sensitive form transmits content over HTTPS.

Output

```
Page : /login
Destination Page: /login

Page : /register
Destination Page: /register
```

To see debug logs, please visit individual host

Vulnerability: Web Server Transmits Cleartext Credentials

The Core Issue

The web application is transmitting sensitive user data—specifically passwords from the login and registration pages—over an unencrypted HTTP connection. Because the connection lacks SSL/TLS encryption, the passwords travel across the network as plain, readable text.

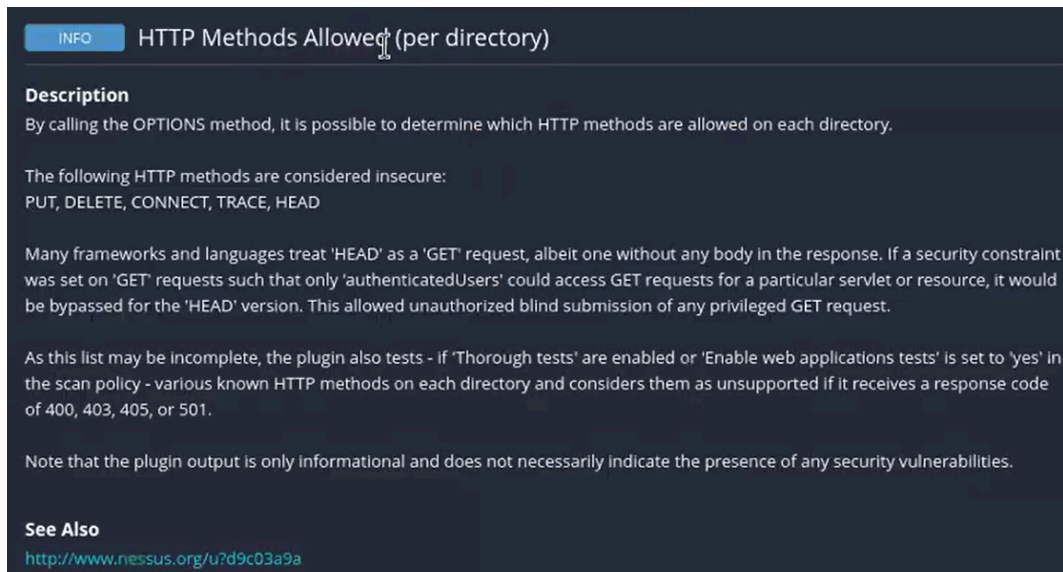
Real-World Scenario

If a user logs into this application while connected to a shared or public network (like a hotel Wi-Fi or even a poorly secured internal office network), an attacker can easily intercept the network traffic using basic packet-sniffing software. Since the password is not encrypted, the attacker can read the credentials immediately and use them to hijack the user's account.

Remediation Steps

To resolve this, the communication channel between the user's browser and the web server must be encrypted.

- **Install an SSL/TLS Certificate:** Ensure a valid certificate is installed and configured on the web server to enable secure HTTPS connections.
- **Enforce HTTPS Redirection:** Configure the web server to automatically redirect all plain HTTP traffic (port 80) to secure HTTPS traffic (port 443).
- **Update Form Actions:** Verify that the HTML `<form>` tags on the `/login` and `/register` pages are explicitly set to submit data to `https://` URLs.
- **Enable HSTS:** Implement the HTTP Strict Transport Security (HSTS) header to instruct web browsers to only communicate with the application over a secure connection moving forward.



Finding: HTTP Methods Allowed (Informational)

The Core Issue

This is an informational finding indicating that the web server is openly broadcasting which HTTP commands (methods) it accepts when asked. While standard commands like `GET` (loading a page) and `POST` (submitting a form) are required for regular browsing, the server is also allowing extended methods such as `PUT`, `DELETE`, `TRACE`, or `HEAD`. Leaving these unnecessary methods enabled increases the server's attack surface.

Real-World Scenario

If the `PUT` or `DELETE` methods are enabled and the directory lacks proper access controls, an attacker could potentially upload a malicious script (like a web shell) directly to your server, or delete critical application files.

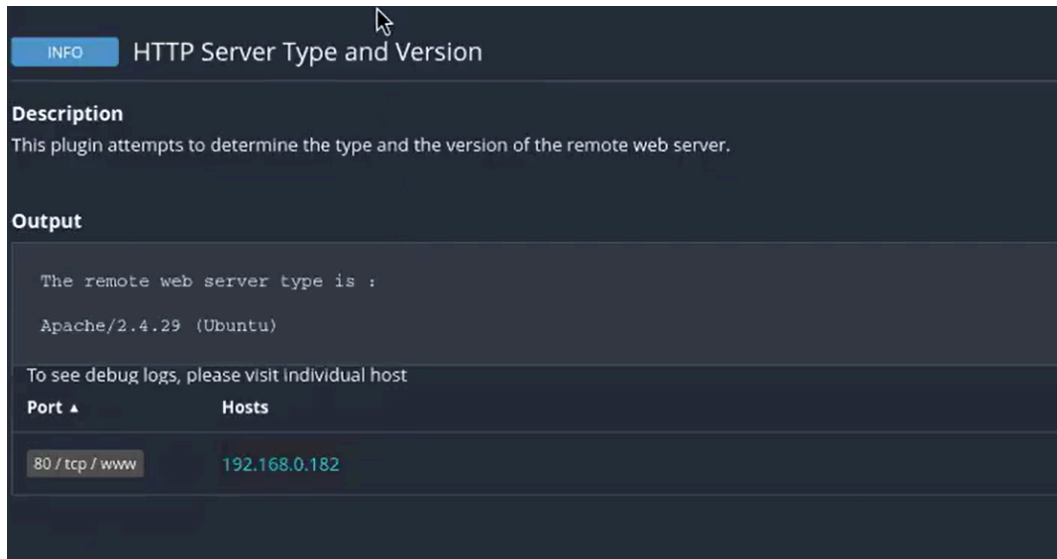
Another subtle attack involves the `HEAD` method. Imagine you have a sensitive admin function protected by a rule that says "Only authenticated users can send `GET` requests here." An attacker could send a `HEAD` request instead. Some poorly configured frameworks will process the `HEAD` request exactly like a `GET` request, effectively bypassing the authentication check and triggering the sensitive function blindly.

Remediation Steps

The best practice is to follow the principle of least privilege by restricting the web server to only accept the HTTP methods strictly necessary for the application to function.

- **Disable Unused Methods:** Configure your web server (e.g., IIS, Apache, Nginx) to explicitly reject methods like `PUT`, `DELETE`, `TRACE`, `TRACK`, and `CONNECT` globally. Typically, standard web applications only need `GET`, `POST`, and occasionally `OPTIONS`.

- **Secure API Endpoints:** If your application relies on REST APIs that legitimately require `PUT` or `DELETE` commands, ensure these methods are only allowed on those specific API directories, and verify that strict authentication is enforced before the server processes them.
- **Review Access Controls:** Ensure that your security filters and authentication checks apply to *all* HTTP methods (including `HEAD`), rather than being mapped exclusively to `GET` or `POST` requests.



Finding: HTTP Server Type and Version Disclosure (Informational)

The Core Issue

This is an informational finding, often referred to as "banner grabbing" or "server fingerprinting." It is not a direct vulnerability, but rather a configuration issue. The web server is openly broadcasting its exact software type and version number (in this case, Apache 2.4.29 on Ubuntu) in its HTTP response headers to anyone who connects to it.

Real-World Scenario

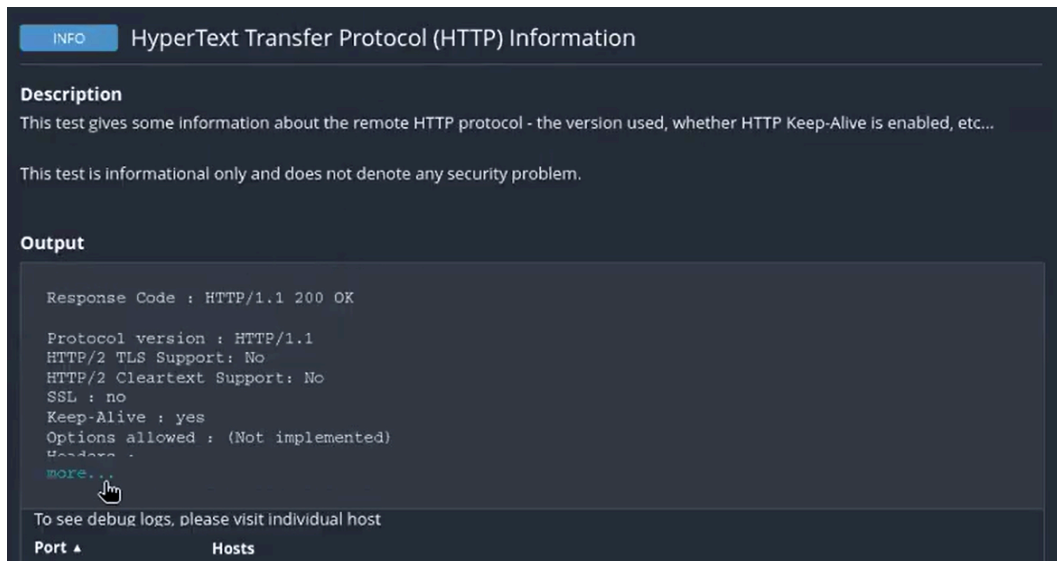
When a hacker begins targeting a network, their first step is reconnaissance. If a server announces exactly what software it is running, the attacker doesn't have to guess or use noisy scanning techniques to figure it out. They can simply search public vulnerability databases for known exploits that specifically affect "Apache 2.4.29." By handing over the version number, you give the attacker a shortcut to finding the right weapon to use against your system.

Remediation Steps

While hiding the version number doesn't fix underlying bugs, it is a standard security best practice that slows down attackers and prevents automated scripts from easily profiling your server.

Since the output identifies this as an Apache server, you should modify the server's configuration file (usually `apache2.conf` or `httpd.conf`) to restrict the information it shares:

- **Change ServerTokens:** Add or update the directive to `ServerTokens Prod`. This tells the server to only identify itself as "Apache" in the HTTP headers, stripping away the version number and OS details.
- **Change ServerSignature:** Add or update the directive to `ServerSignature Off`. This prevents the server from displaying its version information on default error pages (like 404 Not Found pages).
- **Restart the Service:** Once the configuration is saved, restart the Apache service for the changes to take effect.



Finding: HyperText Transfer Protocol (HTTP) Information (Informational)

The Core Issue

This is not a vulnerability; it is an informational finding. The scanner is simply taking notes on how your web server is configured to handle basic web traffic. The output shows that the server is using the older HTTP/1.1 protocol, has "Keep-Alive" connections enabled, and is operating without SSL encryption.

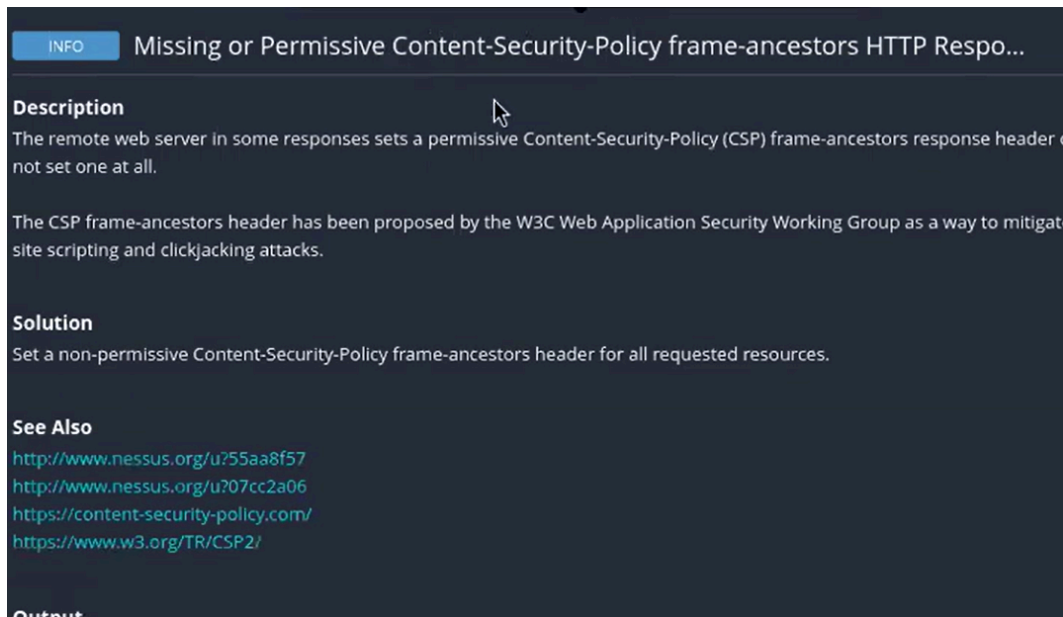
Real-World Scenario

When an attacker scopes out a target, they first gather standard configuration details like this to understand the environment. Seeing that the server is running on plain HTTP/1.1 and explicitly shows "SSL: no" acts as a green light for an attacker. It confirms that the connection is entirely unencrypted, meaning they can easily intercept traffic on the network to capture passwords or sensitive data.

Remediation Steps

Since this is just an informational flag identifying server settings, there is no specific vulnerability to patch. However, the details provided in the output highlight configurations you should improve:

- **Implement SSL/TLS:** The scanner explicitly noted the absence of SSL. As mentioned in previous findings, you need to enable HTTPS across the server to encrypt all web traffic.
- **Review Keep-Alive Limits:** "Keep-Alive" is enabled, which is great for server performance because it reuses connections. Just ensure your server configuration includes strict timeout limits so attackers cannot tie up your server's resources by intentionally holding connections open (a technique used in Slowloris denial-of-service attacks).
- **Upgrade to HTTP/2:** If your server environment supports it, consider upgrading from HTTP/1.1 to HTTP/2. It provides better performance, more efficient connection handling, and generally enforces stricter modern security standards.



Finding: Missing or Permissive Content-Security-Policy frame-ancestors (Informational)

The Core Issue

Your web server is not strictly controlling which other websites are allowed to embed your web pages inside an iframe. Either the `Content-Security-Policy` (CSP) header is completely missing, or the `frame-ancestors` rule within it is set too loosely (for example, allowing any external site to frame it).

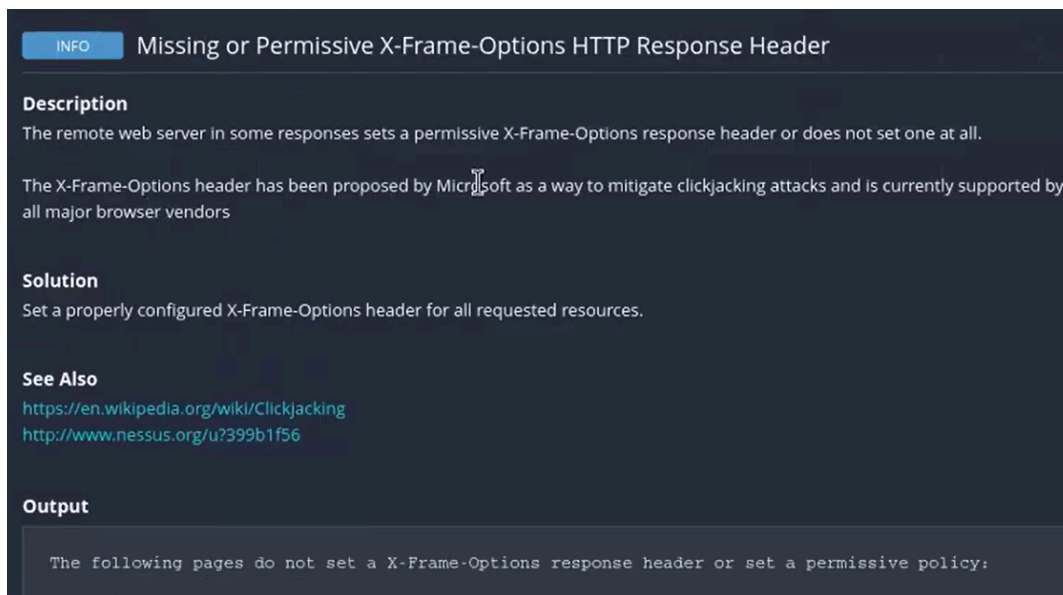
Real-World Scenario

This finding is the exact root cause of Clickjacking (similar to the first vulnerability we looked at). If an attacker can embed your application inside a hidden iframe on their own malicious website, they can trick users into performing unwanted actions. For instance, a user thinks they are clicking a "Play Video" button on the attacker's site, but their click actually registers on a hidden "Transfer Funds" or "Delete Account" button on your embedded application.

Remediation Steps

To fix this, you need to configure your web server or application framework to enforce a strict CSP header that explicitly dictates framing rules across all pages.

- **Block all framing (Recommended for most apps):** If your site never needs to be embedded elsewhere, configure the header as:
`Content-Security-Policy: frame-ancestors 'none';`
- **Allow only your own domain:** If you use iframes internally within your own application, use:
`Content-Security-Policy: frame-ancestors 'self';`
- **Allow specific trusted partners:** If a third party absolutely must frame your site, specify their exact domain and nothing else:
`Content-Security-Policy: frame-ancestors https://trusted-partner.com;`



Finding: Missing or Permissive X-Frame-Options HTTP Response Header (Informational)

The Core Issue

This finding is closely related to the Clickjacking vulnerability we looked at earlier. It specifically highlights that the web server is failing to consistently send the `X-Frame-Options` security header across all of its web pages. Without this header, the server isn't explicitly telling web browsers whether or not it is safe to load the site inside a frame.

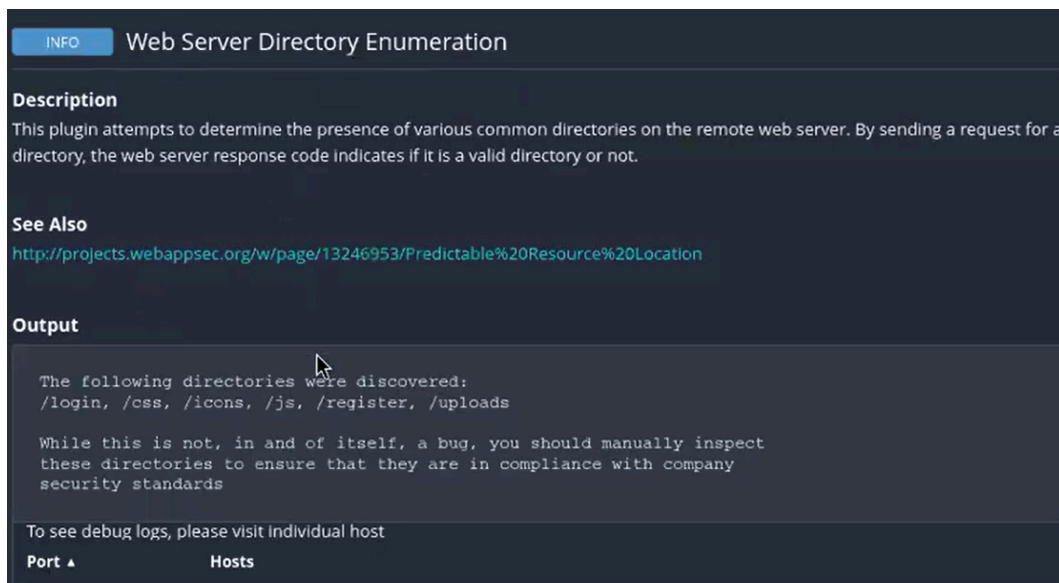
Real-World Scenario

Security is only as strong as its weakest link. Even if you secure your main login page, if an attacker finds just one sensitive page—like a "Delete Profile" or "Change Email" page—that is missing this header, they can target it. They will embed that specific, unprotected page into their own malicious website using an invisible `<iframe>`. When a user interacts with the attacker's site, they are secretly clicking buttons on your unprotected page.

Remediation Steps

You need to enforce a strict framing policy globally across the entire web application so that no pages are left exposed.

- **Configure the Web Server:** Modify your web server configuration (e.g., Apache's `apache2.conf` or Nginx's `nginx.conf`) to ensure the `X-Frame-Options` header is appended to every HTTP response.
- **Set the Correct Value:** * Use `X-Frame-Options: DENY` if your site never needs to be embedded in a frame anywhere.
 - Use `X-Frame-Options: SAMEORIGIN` if you only want to allow pages on your own exact domain to frame each other.
- **Consider CSP:** For modern browsers, implementing the `Content-Security-Policy: frame-ancestors` header alongside `X-Frame-Options` provides the most robust defense.



Finding: Web Server Directory Enumeration (Informational)

The Core Issue

This is an informational finding, not a direct vulnerability. The scanner essentially knocked on a bunch of common doors (folder names) to see which ones your web server would answer. It successfully discovered the existence of several directories, including `/login`, `/register`, and notably, `/uploads`.

Real-World Scenario

When hackers target a website, they don't just look at the visible links. They run automated brute-force tools (like DirBuster or Gobuster) that rapidly guess thousands of hidden directory names. By discovering the exact location of your `/uploads` folder, an attacker now has a specific target. If they find a weakness in your file upload process, they know exactly where to go to execute the malicious script they just uploaded, potentially taking full control of the server.

Remediation Steps

Because web servers inherently need directories to function, you cannot "patch" this finding. However, you must ensure these discovered areas are locked down tightly:

- **Disable Directory Browsing:** Ensure your web server (Apache, Nginx, IIS) is configured to prevent directory listing. If a user navigates to `yourwebsite.com/uploads/` without specifying a file, the server should return a "403 Forbidden" error rather than displaying a list of all files in that folder.
- **Review Folder Permissions:** Double-check the permissions on the `/uploads` directory. It should be strictly configured to store files, but absolutely forbidden from *executing* scripts (like `.php` or `.py` files) from within that directory.
- **Implement Rate Limiting:** Configure your Web Application Firewall (WAF) to detect and block IP addresses that are making an unnaturally high volume of requests for random directories (which indicates an attacker is running an enumeration tool).
- **Audit for Sensitive Data:** Manually verify that no internal company documents, old backup files, or `.git` repositories are accidentally stored in any publicly accessible directories.

INFO

Web Server Allows Password Auto-Completion

Description

The remote web server contains at least one HTML form field that has an input of type 'password' where 'autocomplete' is not set to 'off'.

While this does not represent a risk to this web server per se, it does mean that users who use the affected forms may have their credentials saved in their browsers, which could in turn lead to a loss of confidentiality if any of them use a shared host or if their machine is compromised at some point.

Solution

Add the attribute 'autocomplete=off' to these fields to prevent browsers from caching credentials.

Output

```
Page : /login
Destination Page: /login

Page : /register
Destination Page: /register
```

Finding: Web Server Allows Password Auto-Completion (Informational)

The Core Issue

The web application does not explicitly tell user web browsers to disable password caching. Because the `autocomplete` attribute is missing (or not set to 'off') on the password fields, web browsers will naturally prompt users to save their credentials and will automatically fill them in during future visits.

Real-World Scenario

An employee logs into the application using a shared computer, such as a terminal in a hotel business center, a public library, or a shared hot-desk workstation in an office. They carelessly allow the browser to save their password. When the next person sits down at that exact same computer and navigates to the login page, the browser automatically populates the username and password fields. The second person can now click "Log In" and instantly gain unauthorized access to the first person's account without ever knowing the actual password.

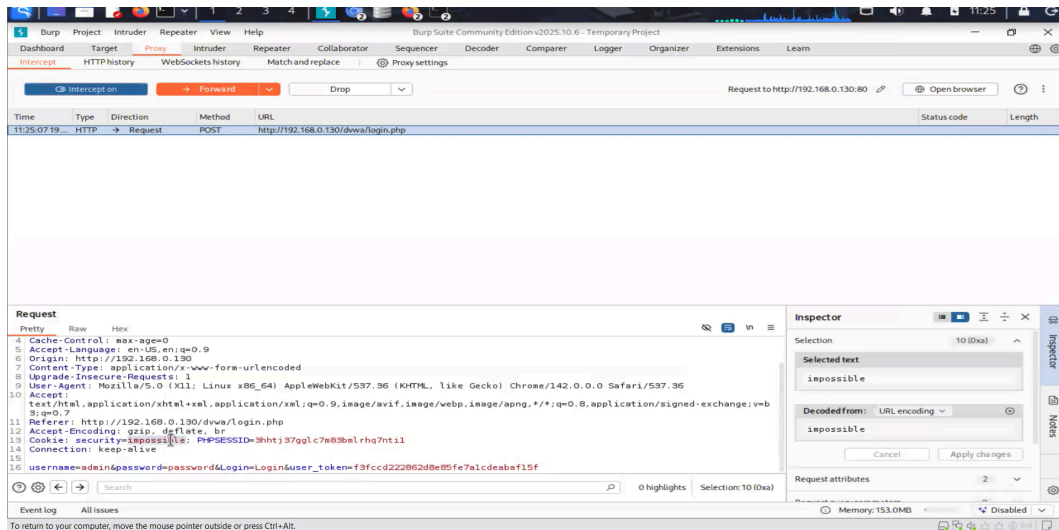
Remediation Steps

To fix this, you need to modify the HTML code of the application's forms to instruct browsers not to save the input data.

- **Update HTML Attributes:** Add the `autocomplete="off"` attribute directly to the `<input type="password">` tags on both the `/login` and `/register` pages.
- **Update Form Tags:** For broader coverage, you can also add `autocomplete="off"` to the main `<form>` tag that contains the sensitive input fields.

Note: While some modern web browsers occasionally ignore this directive in favor of their built-in password managers, adding the attribute remains a standard security best practice and fulfills compliance checklist requirements.

Burp Suite:



you are easily able to modify things using Burp suit.

Vulnerability: Unencrypted Transmission of Login Credentials (HTTP)

The Core Issue

This Burp Suite intercept provides visual proof of the cleartext transmission issue identified earlier. The screenshot shows a POST request to the `/dvwa/login.php` page where the user's credentials (`username=admin` and `password=password`) are sent entirely in plain text within the body of the request. Because the URL uses standard `http://` rather than a secure connection, there is no encryption protecting this data in transit.

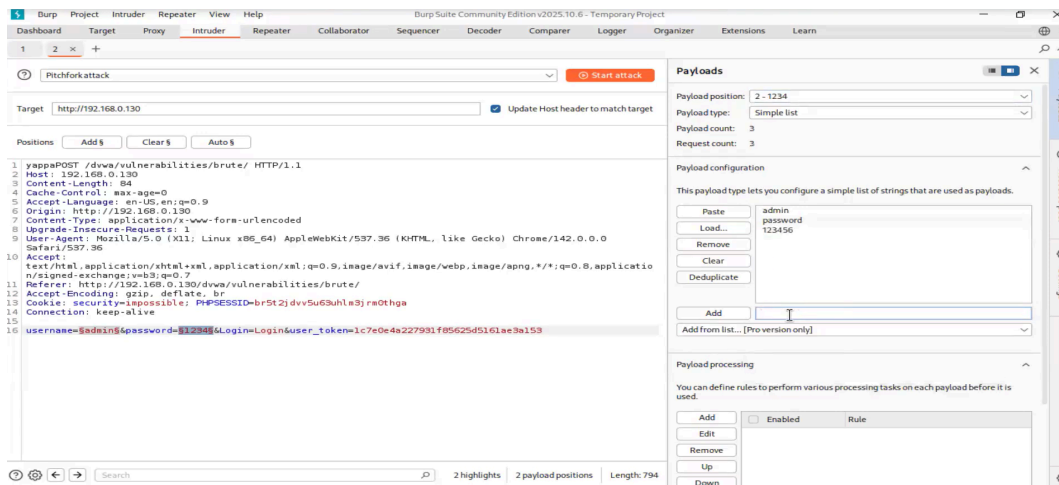
Real-World Scenario

Imagine an administrator logging into this portal while connected to a compromised corporate network or a public Wi-Fi hotspot. An attacker positioned on the same network does not need to hack the server itself; they simply run a network sniffer (like Wireshark) or a proxy (like Burp Suite) to silently capture the traffic. When the administrator clicks "Login," the attacker captures the exact raw request shown in the screenshot, instantly compromising the administrator's username and password.

Remediation Steps

To prevent attackers from intercepting these credentials, the network traffic must be encrypted.

- **Implement HTTPS:** Install an SSL/TLS certificate on the web server and configure it to accept secure connections.
- **Force Secure Connections:** Configure the web application or server to automatically redirect all HTTP traffic to HTTPS.
- **Set Secure Cookie Flags:** Ensure that session cookies (like the `PHPSESSID` seen in the request headers) are configured with the `Secure` flag so they are only ever transmitted over encrypted channels.



Vulnerability: Susceptibility to Brute Force Attacks (Lack of Rate Limiting)

The Core Issue

The web application's authentication mechanism does not limit the number of failed login attempts. As demonstrated in the Burp Suite Intruder screenshot, an attacker can automate the submission of login requests using payloads of various usernames and passwords. Because the server does not block or slow down these repeated requests, it allows continuous guessing of credentials until a valid combination is found.

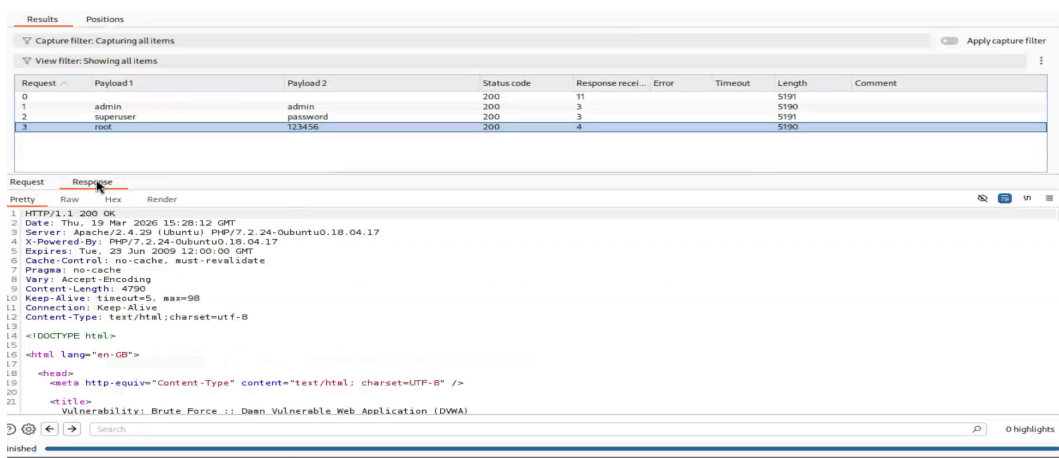
Real-World Scenario

An attacker downloads a list of commonly used passwords or a dump of credentials leaked from another website breach. They configure an automated tool to target your application's login page, rapidly testing thousands of username and password combinations per minute. Without any defenses in place, the tool runs uninterrupted. Within minutes or hours, the tool successfully guesses an employee's weak password, granting the attacker unauthorized access to sensitive company data or administrative controls.

Remediation Steps

To defend against automated credential guessing, implement multiple layers of protection on all authentication endpoints:

- **Implement Account Lockout:** Temporarily disable an account after a set number of consecutive failed login attempts (e.g., lock the account for 15 to 30 minutes after 5 failed tries).
- **Deploy Rate Limiting:** Restrict the total volume of login requests that can originate from a single IP address within a specific timeframe to slow down automated scripts.
- **Integrate CAPTCHA:** Require users to solve a CAPTCHA challenge after two or three failed login attempts. This effectively blocks automated bots while keeping the application accessible to legitimate users who may have just mistyped their password.
- **Enforce Password Complexity:** Require users to create strong, complex passwords (a mix of uppercase, lowercase, numbers, and symbols) to make dictionary-based guessing attacks mathematically impractical.



Vulnerability: Lack of Brute-Force Protection

The Core Issue

The web application's login page does not restrict or monitor the number of failed login attempts. This allows an attacker to use automated tools (like Burp Suite Intruder, as shown in the screenshot) to rapidly test hundreds or thousands of username and password combinations without the server blocking or slowing down the requests.

Real-World Scenario

An attacker discovers your company's employee login portal. They load a list of known employee email addresses and a dictionary file containing millions of common passwords. Because there is no limit on login attempts, they launch an automated bot that fires off 50 login requests per second. Eventually, the bot successfully guesses an employee's weak password (like "Welcome2026!"), giving the attacker immediate, unauthorized access to the internal network.

Remediation Steps

To fix this, you must implement mechanisms that disrupt and prevent automated guessing attacks:

- **Implement Account Lockout:** Configure the application to temporarily lock a user account (e.g., for 15 minutes) after a small number of consecutive failed login attempts (such as 3 to 5 attempts).
- **Apply Rate Limiting:** Restrict the number of login requests that a single IP address can make within a specific timeframe. This drastically slows down automated tools, making brute-force attacks impractical.
- **Require Multi-Factor Authentication (MFA):** Enforce a second form of verification (like an authenticator app code or hardware token). With MFA enabled, an attacker cannot access the account even if they successfully guess the password.
- **Integrate CAPTCHA:** Introduce a CAPTCHA challenge on the login page (or trigger it after one or two failed attempts) to verify that the login requests are being generated by a real human rather than an automated script.

Summary Table

Category	Key Details
Project Overview	• The project establishes a controlled, simulated VAPT lab environment for practical cybersecurity training .

	• It focuses on both Network VAPT and Web Application VAPT, utilizing the OWASP Top 10 as a guiding framework.
Lab Infrastructure	• Hypervisor: VMware . • Attacker Machine: Kali Linux, pre-configured with security tooling . • Vulnerable Server: Ubuntu Server hosting a vulnerable Laravel application.
Target Applications	• Metasploitable2: A Linux VM configured with network-level vulnerabilities . • DVWA: A deliberately insecure PHP/MySQL application . • OWASP Juice Shop & WebGoat: Modern vulnerable web applications.
Primary Tools Used	• Network & Scanning: Nmap, Nikto, Nessus . • Web Traffic Analysis: Burp Suite & OWASP ZAP . • Exploitation & Brute-Force: Metasploit, SQLMap, Hydra.
Key Vulnerabilities Identified	• Clickjacking: Web applications missing X-Frame-Options or CSP frame-ancestors headers . • Cleartext Credentials: Transmission of sensitive login data over unencrypted HTTP connections . • Directory Enumeration: Exposure of common directories like <code>/uploads</code> , <code>/login</code> , and <code>/register</code> . • Brute Force Susceptibility: Lack of rate limiting or account lockout mechanisms on login pages.

Conclusion

Summary of findings and overall security posture

The vulnerability assessment and penetration testing (VAPT) engagement revealed a highly insecure overall security posture, which is consistent with the deployment of unhardened and deliberately vulnerable target applications. The environment is highly susceptible to several critical and medium-severity flaws that align with the OWASP Top 10. Key findings heavily impact the confidentiality and integrity of user data, including:

- **Broken Authentication:** The application transmits user credentials in cleartext over unencrypted HTTP connections, allowing for easy network interception. Furthermore, the lack of rate-limiting or account lockout mechanisms leaves the login portals completely defenseless against automated brute-force attacks.
- **Security Misconfigurations:** The web server fails to implement essential security headers (such as `X-Frame-Options` and `Content-Security-Policy`), leaving the application vulnerable to Clickjacking or UI redress attacks. Additionally, the server freely discloses its software version (Apache 2.4.29) and allows insecure HTTP methods, significantly lowering the barrier for attackers during the reconnaissance phase.
- **Information Disclosure:** Directory enumeration exposed sensitive operational folders such as `/uploads` and `/login` , providing an attacker with a clear map of the application's attack surface.

What was learned from the engagement

This project provided invaluable hands-on experience in the practical application of offensive security techniques. Key takeaways include:

- **Lab Deployment:** Successfully architecting a safe, isolated, and virtualized network environment using VMware to host an attacker machine (Kali Linux) and multiple vulnerable targets (Ubuntu Server, DVWA, Metasploitable2).
- **Tool Proficiency:** Gaining practical proficiency with industry-standard security tools. This included using Nmap for network discovery , configuring Nessus for automated vulnerability scanning , and utilizing Burp Suite Intruder to intercept traffic and execute automated brute-force payload attacks.

- **Vulnerability Remediation:** Understanding that identifying a flaw is only half the battle. This engagement highlighted the critical importance of remediation strategies, such as enforcing HTTPS, configuring Web Application Firewalls (WAFs), and implementing strict server-side validation and HTTP headers.

Limitations of your testing

While comprehensive within its scope, this assessment had several inherent limitations:

- **Simulated Environment:** The testing was conducted in a controlled, offline virtual environment. This scenario lacks the complex, layered security controls—such as Intrusion Detection/Prevention Systems (IDS/IPS), Web Application Firewalls (WAFs), or load balancers—that are standard in modern production networks.
- **Automated Scanning Reliance:** Portions of the assessment relied on automated tools like Nessus. While highly effective for discovering known vulnerabilities and missing patches, automated scanners can produce false positives or miss complex, multi-step business logic flaws that require deep manual testing.
- **Scope Restrictions:** The engagement was limited to specific, deliberately vulnerable applications configured for this lab. It did not include testing against custom-developed, production-grade source code or advanced social engineering tactics.

Future improvements

To build upon the foundation established in this VAPT project, future iterations and engagements could incorporate the following improvements:

- **Implementation of Defensive Mechanisms:** Future labs should transition into a "Purple Team" exercise by actively deploying and configuring defensive tools (like ModSecurity WAF or Suricata IDS) on the target servers to practice detection and evasion techniques.
- **Advanced Exploitation:** Expanding the scope to include post-exploitation activities, such as network pivoting, privilege escalation, and Active Directory exploitation, to simulate a full-scale persistent threat.
- **DevSecOps Integration:** Integrating automated security testing (SAST/DAST) directly into a CI/CD pipeline for the lab's web applications to demonstrate how vulnerabilities can be caught and mitigated during the software development lifecycle, rather than just in post-deployment audits.

References

Official Tools and Manuals

- **Nmap Security Scanner:** Official site at <https://nmap.org> and source repository at <https://github.com/nmap/nmap>.
- **OWASP ZAP (Zed Attack Proxy):** Official homepage at <https://www.zaproxy.org/>.
- **Burp Suite:** Kali Linux package repository at <https://gitlab.com/kalilinux/packages/burpsuite>.
- **Tenable Nessus:** Download repository at https://www.tenable.com/downloads/api/v2/pages/nessus/files/Nessus-10.11.3-debian10_amd64.deb.
- **wndoc:** Pentesting report generator documentation at <https://pwndoc.github.io/pwndoc/#/>.

Target Environments & Lab Assets

- **Vulnerable Laravel Application:** Source code and deployment guide available at <https://github.com/dennisehcs/vulnerable-laravel-app>.

- **Ubuntu Server:** Official releases downloaded from <https://releases.ubuntu.com/>.
- **Ubuntu Documentation:** Official management and support manuals at <https://help.ubuntu.com>.

Vulnerability References & Security Frameworks

- **OWASP Clickjacking Defense Cheat Sheet:** Mitigation strategies at https://www.owasp.org/index.php/Clickjacking_Defense_Cheat_Sheet.
- **W3C Content Security Policy (CSP):** Official documentation at <https://www.w3.org/TR/CSP2/> and <https://content-security-policy.com/>.
- **Clickjacking Overview:** General vulnerability information at <https://en.wikipedia.org/wiki/Clickjacking>.
- **Nessus Plugin Documentation (Clickjacking & X-Frame-Options)**
- **Nessus Plugin Documentation (Insecure HTTP Methods)**
- **Nessus Plugin Documentation (CSP frame-ancestors)**